

ción de la compañía o de la tienda entera como el sistema es importante. En el sistema TPDV, definiremos “el sistema” como la terminal de punto de venta y su software.

6.12 Casos de uso primarios, secundarios y opcionales

Los casos deberían clasificarse en primarios, secundarios u opcionales. Más adelante, a partir de estas designaciones, clasificaremos nuestro conjunto de casos de uso para establecer prioridades en su desarrollo.

Los **casos primarios de uso** representan los procesos comunes más importantes, como *Comprar productos*.

Los **casos secundarios de uso** representan procesos menores o raros; por ejemplo, *Solicitud de surtir el nuevo producto*.

Los **casos opcionales de uso** representan procesos que pueden no abordarse.

6.13 Casos esenciales de uso comparados con los casos reales de uso

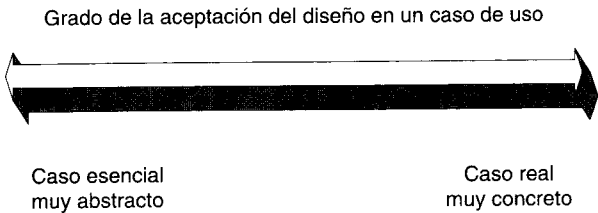


Figura 6.7 Los casos esenciales y reales de uso existen a lo largo de un continuo.

6.13.1 Casos esenciales de uso

Los **casos esenciales de uso** [Constantine97] son casos expandidos que se expresan en una forma teórica que contiene poca tecnología y pocos detalles de implementación; las decisiones de diseño se posponen y se abstraen de la realidad, especialmente las concernientes a la interfaz para el usuario. Un caso de este tipo describe el proceso a partir de sus actividades y motivos esenciales. El grado de abstracción con que se describe existe en un continuo: la descripción puede ser más o menos esencial.

Los casos de alto nivel siempre son de carácter esencial, debido a su brevedad y abstracción.

En seguida se incluye un ejemplo de un caso de *Retiro de efectivo* en un cajero automático, que se expresa en una forma relativamente esencial.

Esencial

Acción de los actores	Respuesta del sistema
1. El Cliente se identifica.	2. Presenta opciones.
3. y así sucesivamente.	4. y así sucesivamente.

La *manera* en que un cliente se identifica cambia con el tiempo —es una decisión de diseño—, pero forma parte del proceso esencial de que la identificación se realice de alguna manera.

Conviene crear casos esenciales de uso al comenzar a investigar los requerimientos, con el propósito de entender mejor el alcance del problema y las funciones necesarias. Este tipo de casos son de gran utilidad porque permiten captar la esencia del proceso y sus motivos fundamentales, sin verse abrumado con detalles de diseño. Suelen también ser correctos durante largo tiempo, ya que excluye las decisiones de diseño y, por lo mismo, su creación favorece la comprensión y el registro de los factores que dan vida a los procesos de un negocio. Una empresa puede recuperar y releer los casos esenciales de uso mucho tiempo después, aplicándolos exitosamente a un nuevo proyecto de desarrollo.

13.2 Casos reales de uso

En cambio, un **caso real de uso** describe concretamente el proceso a partir de su diseño concreto actual, sujeto a las tecnologías específicas de entrada y de salida, etc. Cuando se trata de la interfaz para el usuario, a menudo ofrece presentaciones de pantalla y explica la interacción con los artefactos. A continuación se incluye el caso *Retiro de efectivo* expresado en una forma relativamente *real*.

Real

Acción de los actores	Respuesta del sistema
1. El Cliente introduce su tarjeta.	2. Pide el número de identificación personal (NIP).
3. Introduce el NIP con un teclado numérico.	4. Muestra el menú de opciones.
5. y así sucesivamente.	6. y así sucesivamente.

Nótese que la acción esencial del “Cliente se identifica a sí mismo” del caso de uso se realizó ahora concretamente en la serie de acciones comenzando con “El Cliente introduce su tarjeta”.

En teoría, los casos reales de uso se crean durante la fase de diseño en un ciclo de desarrollo, por ser un artefacto del diseño. En algunos proyectos se prevén las primeras decisiones de diseño concernientes a la interfaz para el usuario; de ahí la necesidad de crear casos reales en la fase inicial de elaboración. Se recomienda hacerlo en la fase de planeación y elaboración por la aceptación prematura de un diseño y la abrumadora complejidad. No obstante, algunas compañías aceptan un contrato de desarrollo, basándose en las especificaciones de la interfaz para el usuario.

En el capítulo 19 se examinan los casos reales en la aplicación al punto de venta.

6.13.3 El caso esencial de uso de la compra de productos

El caso ampliado de uso *Comprar productos* que ya mencionamos tiende a ser un caso esencial. Obsérvese que la descripción no es muy específica respecto a la realización técnica. El caso está escrito de manera que casi podemos imaginar su aplicabilidad después de cien años o hace cien años, lo cual manifiesta que es esencial.

Esencial	
Acción de los actores	Respuesta del sistema
1. El Cajero registra el identificador en cada producto. Si hay más de un producto igual, el Cajero puede introducir de igual manera la cantidad.	2. Determina el precio del producto y agrega la información sobre él a la actual transacción de venta. Aparecen la descripción y el precio del producto actual.
3. y así sucesivamente.	4. y así sucesivamente.

6.13.4 El caso real de uso de la compra de productos

A diferencia de una versión esencial del caso de uso, una versión real se compromete con el diseño; una versión completa de ella se explicará en un capítulo posterior. En el siguiente ejemplo de la versión real, nótese la decisión de utilizar un código universal de producto (CUP) con el identificador del producto¹ y una interfaz gráfica para el usuario.

¹ En una fase temprana del análisis no conviene tomar algunas decisiones, como la de utilizar un código universal de producto (CUP) en el caso esencial de uso. El análisis y diseño esencial y real son términos a lo largo de un continuo de abstracción más que extremos. He aquí lo más importante: siempre que se adopta un compromiso durante la fase de análisis, existe la posibilidad de un error prematuro de diseño, sobrecarga de información y menor flexibilidad.

Real	
Acción de los actores	Respuesta del sistema
1. En cada producto, el Cajero teclea el Código Universal de Productos (CUP) en el campo de entrada del CUP de la Ventana1. Después oprime el botón "Introducir producto" con el ratón u oprimiendo la tecla <Enter>.	2. Muestra el precio del producto y agrega la información sobre él a la actual transacción de venta. La descripción y el precio del producto actual se muestran en el cuadro de Texto 2 de la Ventana1.
3. etcétera.	4. etcétera.

6.14 Sobre la notación

6.14.1 Asignación de nombre a los casos de uso

Al caso de uso se le asigna un nombre que comience con un verbo para subrayar que se trata de un proceso. Por ejemplo:

- Comprar productos
- Introducir un pedido

6.14.2 Inicio de un caso expandido de uso

Comience un caso expandido con el siguiente esquema:

1. Este caso comienza cuando <Actor> <inicia un evento>

Por ejemplo:

1. Este caso de uso comienza cuando un Cliente llega a un TPDV con productos que desea comprar.

De este modo se estimula una identificación clara del actor y del evento iniciadores.

6.14.3 Puntos sobre la decisión de notación y sobre la ramificación

Un caso de uso puede contener puntos de decisión. Por ejemplo, en *Comprar productos*, el cliente puede optar por pagar con efectivo, a crédito o con cheque en el momento del pago. Si una de estas trayectorias de decisión es un caso muy representativo y si las otras alternativas son raras, inusuales o excepcionales, el caso típico

deberá ser el único acerca del cual se escribe en el *Curso normal de los eventos* y las opciones han de escribirse en la sección titulada *Alternativas*.

Pero en ocasiones el punto de decisión representa opciones cuya probabilidad es relativamente igual y normal; esto sucede con los tipos de pago en efectivo, con tarjeta de crédito y con cheque. En este caso se utiliza la siguiente estructura notacional:

- 1. En la sección principal *Curso normal de los eventos*, indique las ramas de las subsecciones.
- 2. Escriba una subsección en cada rama, utilizando otra vez un *Curso normal de los eventos*. Inicie el evento numerando en 1 cada sección.
- 3. Si las subsecciones tienen opciones, escríbalas en una sección de alternativas de cada subsección.

Sección: principal

Curso normal de los eventos

Acción de los actores	Respuesta del sistema
1. Este caso de uso comienza cuando un Cliente llega a la caja TPDV con los productos que comprará.	
2. (Excluidos los pasos intermedios)...	
3. El Cliente escoge el tipo de pago:	
a. Si paga en efectivo, consúltese la sección <i>Pago en efectivo</i> .	
b. Si paga a crédito, consúltese la sección <i>Pago con tarjeta de crédito</i> .	
c. Si paga con cheque, consúltese la sección <i>Pago con cheque</i> .	
	4. Registra la venta terminada.
	5. Imprime un recibo.
6. El Cajero le entrega el recibo al Cliente.	
7. El Cliente se marcha con los productos comprados.	

Sección: Pago en efectivo

Curso normal de los eventos

Acción de los actores	Respuesta del sistema
1. El Cliente da un pago en efectivo —el “efectivo ofrecido”—, posiblemente mayor que el total de la venta.	
2. El Cajero registra el efectivo ofrecido.	3. Muestra al Cliente la diferencia.
4. El Cajero deposita el efectivo recibido y extrae la diferencia.	
El Cajero entrega al Cliente el cambio del pago.	

Cursos alternativos

- Línea 4: efectivo insuficiente en la caja para pagar la diferencia. Se pide efectivo al supervisor o se pide al Cliente un pago más cercano al total de la venta.

Sección: pago con tarjeta de crédito

Cursos normales y alternos de la historia de pago con tarjeta de crédito.

Sección: pago con cheque

Cursos normales y alternos de la historia de pago con cheque.

6.15 Casos de uso dentro de un proceso de desarrollo

6.15.1 Pasos de la fase de planeación y elaboración

- 1. Después de haber listado las funciones del sistema, defina la frontera de éste y luego identifique los actores y los casos de uso.

- 2. Escriba todos los casos de uso en el formato de *alto nivel*. Clasifíquelos en primarios, secundarios u opcionales.
- 3. Dibuje un diagrama de caso de uso.
- 4. Relacione los casos de uso y dé ejemplo de las relaciones en el diagrama correspondiente (más adelante se explican las relaciones de los casos).
- 5. Escriba en el formato *esencial expandido* los casos de uso más importantes, influyentes y riesgosos, a fin de entender y estimar mejor la naturaleza y las dimensiones del problema. Para evitar análisis complejos posponga la escritura de la forma esencial expandida de los casos de uso menos importantes hasta los ciclos de desarrollo en que serán abordados.
- 6. En teoría, los casos *reales* deberían posponerse hasta una fase de diseño en el ciclo de desarrollo, porque su creación conlleva decisiones de diseño. Pese a ello, a veces es necesario crear casos reales de uso durante la etapa inicial de los requerimientos si:
 - ▢ Las descripciones concretas facilitan notablemente la comprensión.
 - ▢ Los Clientes exigen especificar sus procesos en esta forma.
- 7. Clasifique los casos de uso (que se expondrán en el siguiente capítulo).

6.15.2 Pasos de la fase del ciclo de desarrollo iterativo

- 1. Fase de análisis: escriba casos esenciales de uso expandidos para los que se han abordado, si todavía no se llevan a cabo.
- 2. Fase de diseño: escriba casos reales de uso para los que están siendo abordados, en caso de que todavía no se realicen.

6.16 Pasos del proceso en un sistema del punto de venta

Explicaremos algunas de las siguientes actividades en capítulos posteriores, ya que requieren una exposición amplia o pueden aplazarse para evitar una sobrecarga de información. Como nuestra meta es adquirir la habilidad de aplicar los casos y no convertirnos en expertos en tiendas, no escribiremos los detalles de todos los casos.

6.16.1 Identifique los actores y los casos de uso

En la aplicación del punto de venta, defina la frontera del sistema que será el sistema de hardware/software, el caso habitual. Un ejemplo de lista de los actores y procesos relevantes a que dan inicio —que no pretende en absoluto ser completa— incluye:

Cajero	Registra los productos Entrega el cambio
Cliente	Compra productos Paga los productos
Gerente	Inicia Cierra
Administrador del sistema	Incorpora nuevos usuarios

6.16.2 Escriba casos de uso en el formato de alto nivel

Una muestra de casos de uso de alto nivel comprende:

Caso de uso: **Comprar productos**
Actores: Cliente (iniciador), Cajero.
Tipo: Primario.
Descripción: Un Cliente llega a una caja con productos que desea comprar. El Cajero registra los productos y obtiene el pago. Al terminar la transacción, el Cliente se marcha con los productos.

Caso de uso: **Inicio de operaciones**
Actores: Gerente.
Tipo: Primario.
Descripción: Un gerente activa una TPDV a fin de prepararla para que la usen los Cajeros. El Gerente comprueba que la fecha y la hora sean correctos; hecho esto, el sistema está listo para que lo utilice el Cajero.

6.16.3 Dibuje un diagrama de casos de uso

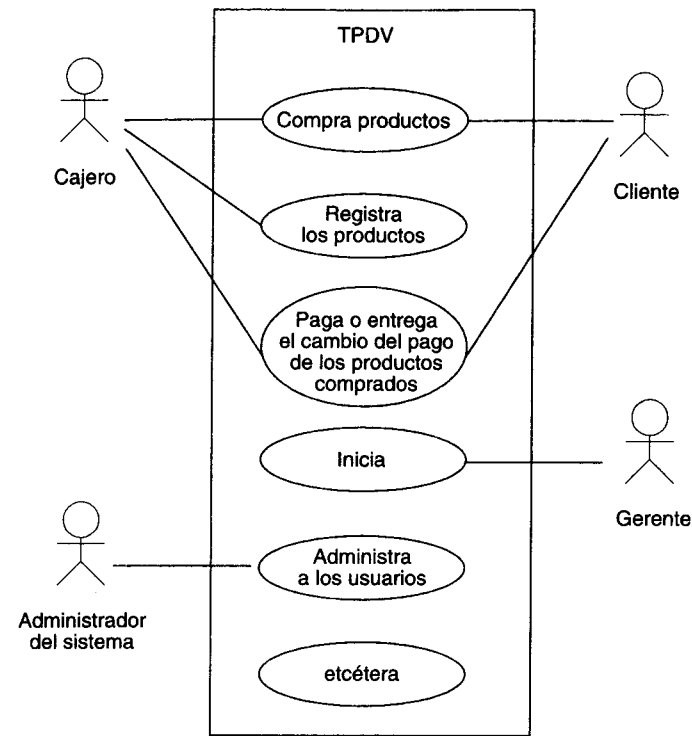


Figura 6.8 Diagrama parcial de un caso de uso, que representa la aplicación TPDV.

6.16.4 Relacione los casos de uso

Este tema lo trataremos en un capítulo posterior.

6.16.5 Escriba algunos casos esenciales expandidos de uso

Entre los casos primarios de uso realmente significativos figuran:

- Comprar productos
- Pagar los productos comprados

Escribir lo anterior en una forma esencial expandida suministrará una mayor información y esclarecimiento de los requerimientos. A continuación se presenta el caso de uso *Comprar productos* en su forma esencial expandida completa:

Casos de uso: comprar productos

Sección: principal

Caso de uso:	Comprar productos
Actores:	Cliente (iniciador), Cajero.
Propósito:	Capturar una venta y su pago.
Resumen:	Un Cliente llega a la caja con productos que desea comprar. El Cajero registra los productos y recibe el pago, que puede ser autorizado. Al terminar la transacción, el Cliente se marcha con los productos.
Tipo:	Primario y esencial.
Referencias cruzadas:	Funciones: R1.1, R1.2, R1.3, R1.7, R1.9, R2.1, R2.2, R2.3, R2.4. Casos de uso: el Cajero debe haber terminado el caso de uso: Registrar.

Curso normal de los eventos

Acción de los actores	Respuesta del sistema
1. Este caso de uso comienza cuando un Cliente llega a la caja TPDV con productos que desea comprar.	
2. El Cajero registra los productos. Si hay más de un producto, también puede introducir la cantidad.	3. Determina el precio del producto y agrega la información sobre él a la actual transacción de venta. Se muestran la descripción y el precio del producto actual.
4. Al terminar la captura de los productos, el Cajero indica a TPDV que terminó la captura de los productos.	5. Calcula y presenta el total de la venta.
6. El Cajero le indica el total al Cliente.	

Curso normal de los eventos

Acción de los actores	Respuesta del sistema
7. El Cliente escoge la forma de pago: a. Si paga en efectivo, véase la sección <i>Pagar en efectivo</i> . b. Si paga con tarjeta de crédito, véase la sección <i>Pagar con tarjeta de crédito</i> . c. Si paga con cheque, véase la sección <i>Pagar con cheque</i> .	8. Registra la venta terminada. 9. Actualiza los niveles de inventario. 10. Genera un recibo.
11. El Cajero entrega el recibo al cliente.	
12. El Cliente se marcha con los productos comprados.	

Cursos alternos

- Línea 2: se introduce un identificador inválido del producto. Indique el error.
- Línea 7: el Cliente no pudo pagar. Cancele la transacción de venta.

Sección: pagar en efectivo

Curso normal de los eventos

Acción de los actores	Respuesta del sistema
1. El Cliente da un pago en efectivo —el “efectivo ofrecido”—, posiblemente mayor que el total de la venta.	

Curso normal de los eventos

Acción de los actores	Respuesta del sistema
2. El Cajero registra el efectivo ofrecido.	3. Presenta la diferencia al Cliente.
4. El Cajero deposita el efectivo recibido y extrae la diferencia.	
El Cajero le entrega el cambio al Cliente.	

Cursos alternos

- Línea 1: el Cliente no tiene suficiente efectivo. Puede cancelar o iniciar otro método de pago.
- Línea 4: la caja no contiene suficiente efectivo para pagar la diferencia. El Cajero pide más efectivo al supervisor o le pide al Cliente otro billete de menor denominación u otra forma de pago.

Sección: pago con tarjeta de crédito

Curso normal de los eventos

Acción de los actores	Respuesta del sistema
1. El Cliente comunica su información de crédito para pagar con tarjeta.	2. Genera una solicitud de pago con tarjeta de crédito y la envía a un Servicio externo de autorización de crédito.
3. El Servicio de autorización de crédito autoriza el pago.	4. Recibe una respuesta aprobatoria de crédito del Servicio de autorización de crédito.
	5. En el sistema de Cuentas por cobrar registra la información sobre el pago con tarjeta de crédito y la respuesta de aprobación. El Servicio de autorización de crédito debe dinero a la Tienda; por tanto, Cuentas por cobrar debe darle seguimiento.
	6. Muestra el mensaje aprobatorio de autorización.

Cursos alternos

- Línea 3: solicitud de crédito negada por el Servicio de autorización de crédito. Proponer otro método de pago.

Sección: pago con cheque

Curso normal de los eventos

Acción de los actores		Respuesta del sistema	
1.	El Cliente extiende un cheque y se identifica.		
2.	El Cajero registra la información sobre la identificación y solicita la autorización del pago con cheque.	3.	Genera una solicitud de pago con cheque y la envía a un Servicio externo de autorización de cheques.
4.	Verifica que el pago haya sido autorizado por el Servicio de autorización de cheques.	5.	Recibe una respuesta aprobatoria del Servicio de autorización de cheques.
		6.	Indica la obtención de la autorización.

Cursos alternos

- Línea 4: verificar solicitud negada por el Servicio de autorización de cheques. Proponer otra forma de pago.

6.16.6 Si es necesario, escriba algunos casos reales de uso

No conviene o no es necesario crear casos de uso real en este momento; este trabajo se realizará durante los ciclos de desarrollo.

6.16.7 Clasifique los casos de uso

Este tema se estudiará en el siguiente capítulo.

6.17 Modelos muestra

Los casos esenciales de alto nivel y los diagramas de casos de uso son miembros del modelo de casos de uso del análisis (figura 6.9).

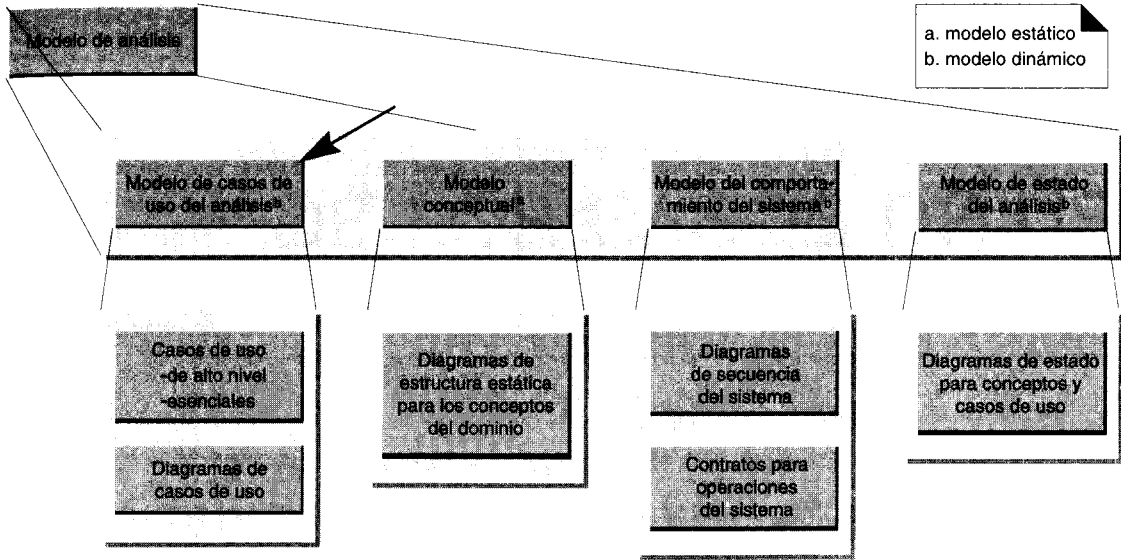


Figura 6.9 Modelo de análisis.

UNIVERSIDAD DE LA REPUBLICA
FACULTAD DE INGENIERIA
DEPARTAMENTO DE
DOCUMENTACION Y BIBLIOTECA
MONTEVIDEO - URUGUAY

CLASIFICACIÓN Y PROGRAMACIÓN DE LOS CASOS DE USO

Objetivos

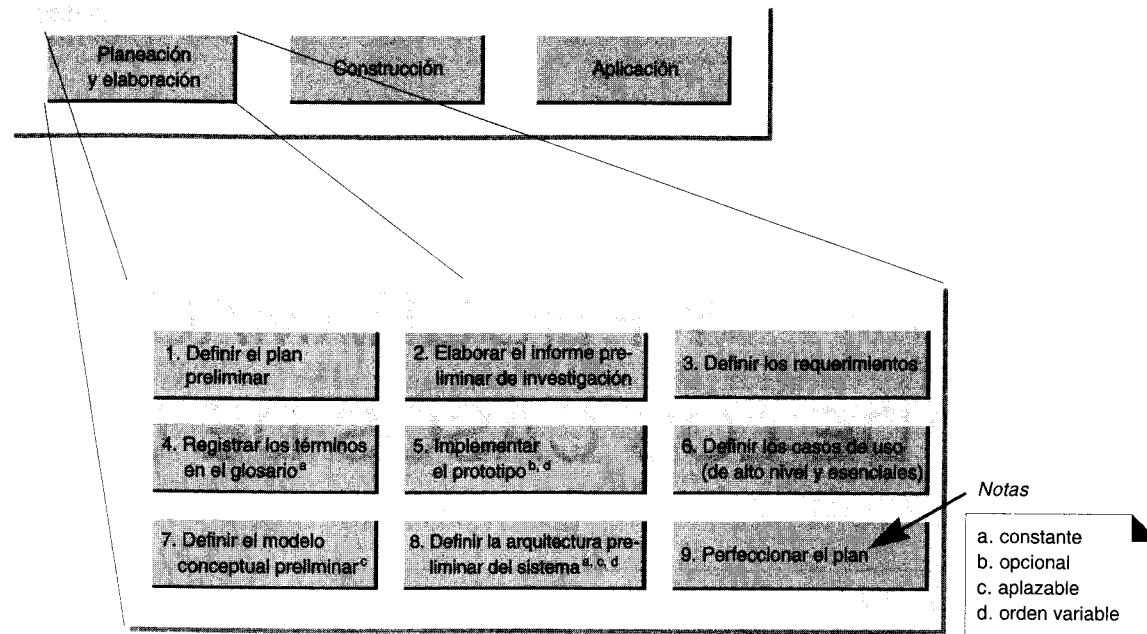
- Clasificar los casos de uso.
- Cuando sea necesario, preparar versiones simplificadas de los casos de uso.
- Asignar los casos de uso a los ciclos de desarrollo.

7.1 Introducción

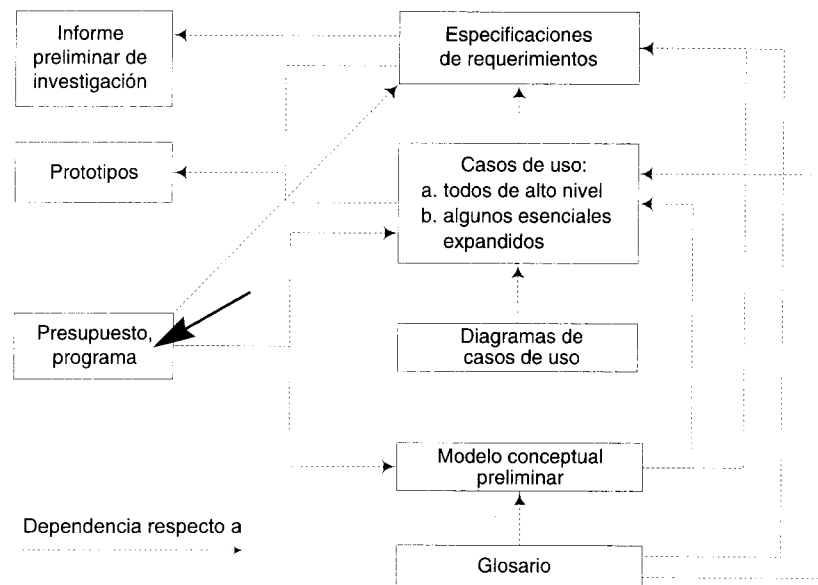
Las especificaciones de los requerimientos y los casos de uso se definen en la Fase de Planeación y Elaboración. Además, puede crearse un Modelo conceptual preliminar y diseñar una arquitectura también preliminar del sistema, aunque estas actividades se pospondrán en nuestro estudio de casos para ofrecer una introducción más gradual a los temas.

Suponiendo que todos los artefactos deseados hayan sido generados (por ejemplo, la especificación de los requerimientos y los casos de uso), el siguiente paso es iniciar la fase de Construcción en el ciclo de desarrollo iterativo y comenzar a implementar el sistema. En un ciclo de vida de desarrollo iterativo, la tarea de llenar los casos de uso se distribuye entre varios ciclos.

En el presente capítulo se estudia la clasificación y la programación o calendarización de los casos de uso. Una vez concluida esta etapa, estaremos listos para comenzar el primer ciclo de desarrollo y examinar a fondo el análisis y diseño orientados a objetos.



Actividades de la fase de planeación y elaboración.



Dependencias de los artefactos respecto a la fase de planeación y elaboración.

7.2 Programación de los casos de uso en los ciclos de desarrollo

7.2.1 Casos de uso y los ciclos de desarrollo

Los ciclos de desarrollo se organizan en torno a los requerimientos de los casos de uso. En otras palabras, se asigna un ciclo para implementar uno o más casos de uso o sus versiones simplificadas, cuando el caso íntegro resulta demasiado complejo para abordarlo en un ciclo (figura 7.1).

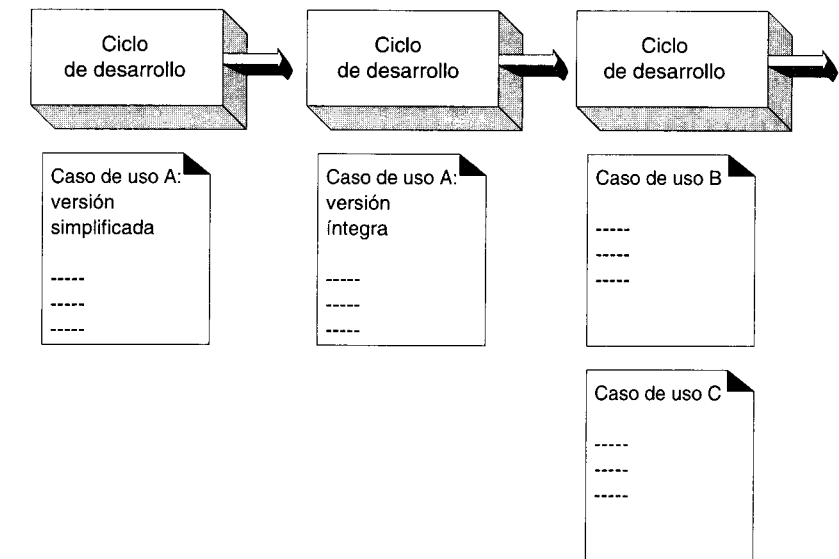


Figura 7.1 Asignación de los casos de uso a los ciclos de desarrollo.

7.2.2 Clasificación de los casos de uso

Es necesario clasificar los casos de uso, y los casos de alto rango han de tratarse al inicio de los ciclos de desarrollo. La estrategia general consiste en escoger primero los casos que influyen profundamente en la arquitectura básica. He aquí algunas de las cualidades que aumenta la clasificación de un caso:

- Tener una fuerte repercusión en el diseño arquitectónico; por ejemplo, incorporar muchas clases a la capa del dominio o requerir servicios de persistencia.
- Con relativamente poco esfuerzo obtener información e ideas importantes sobre el diseño.
- Incluir funciones riesgosas, urgentes o complejas.
- Requerir una investigación a fondo o tecnología nueva y riesgosa.
- Representar procesos primarios de la línea de negocios.
- Apoyar directamente el aumento de ingresos o la reducción de costos.

El esquema taxonómico puede servirse de una clasificación simple y poco rigurosa: alto-mediano-bajo.

El esquema también puede aplicar puntuaciones (posiblemente incrementadas con ponderación), basándose en las cualidades que inciden en la clasificación; por ejemplo:¹

Caso de uso	a	b	c	d	e	f	Suma
Comprar productos	5	3	2	0	5	3	18
y así sucesivamente							

7.3 Clasificación de los casos de uso en la aplicación al punto de venta

Con base en los criterios anteriores de clasificación, a continuación se incluye una clasificación informal y poco rigurosa de los casos de uso de aplicación al punto de venta. De ninguna manera pretendemos dar una lista exhaustiva.

Clasificación	Caso de uso	Justificación
Alto	Comprar productos	Corresponde a los criterios de clasificación más altos.
Mediano	Incorpora nuevos usuarios	Afecta al subdominio de la seguridad.
	Registra los productos comprados	Afecta al subdominio de la seguridad.
	Paga los productos comprados	Proceso importante; afecta a la contabilidad.
Bajo	Pagar	Efecto mínimo en la arquitectura.
	Iniciar	La definición depende de otros casos de uso.
	Cerrar	Efecto mínimo en la arquitectura.

7.4 El caso de uso de arranque

Prácticamente todos los sistemas cuentan con un *Caso de uso de inicio o arranque*. Aunque tal vez no ocupe un nivel alto conforme a otros criterios, es preciso estudiar al menos una versión simplificada de él, al principiar el ciclo de desarrollo para presentar la inicialización supuesta en otros casos. En cada ciclo de desarrollo, éste se

¹ El esquema no es completo, pues tan sólo pretende ofrecer sugerencias.

realizó incrementalmente para satisfacer las necesidades de arranque de otros casos. No vamos a presentar de manera explícita la programación de los pasos de este caso de uso y supondremos que siempre se desarrolla en forma implícita como se necesita.

7.5 Programación de los casos de uso en la aplicación del punto de venta

A partir de la clasificación, el caso *Comprar productos* debería incluirse en el primer ciclo de desarrollo. Como hemos visto, también puede abordarse una versión simple de *Inicio* para soportar los otros casos de uso.

7.5.1 Creación de versiones múltiples de los casos de uso complejos

Siempre que se asigne un caso de uso, es necesario estimar si es posible resolverlo íntegramente en el lapso limitado de un ciclo (cuatro semanas, por ejemplo) o si el trabajo ha de ser distribuido en varios ciclos. En este caso, *Comprar productos* es extremadamente complejo y quizá requiere cinco o más ciclos, suponiendo que cada uno tendrá una duración de cuatro semanas exactamente. Se supone una estrategia de programación de **duración o tiempo fijo**, en la cual al ciclo de desarrollo se le establece un plazo fijo.

En esta situación el caso se redefine a partir de varias versiones de él, que van abarcando requerimientos cada vez más exhaustivos. Cada versión se limita a incluir lo que se estima una cantidad razonable de trabajo dentro de los confines de la duración fija del ciclo (digamos cuatro semanas). Por ejemplo:

- Comprar productos: versión 1 (pagos en efectivo, sin actualizaciones de inventario, ...)
- Comprar productos: versión 2 (permitir cualquier tipo de pago)
- Comprar productos: versión 3 (versión completa, incluyendo entre otras cosas las actualizaciones del inventario, ...)

Las versiones anteriores se distribuyen después a lo largo de una serie de ciclos de desarrollo, junto con otros casos de uso.

7.5.2 Asignación de los casos de uso

Si nos basamos en la clasificación de los casos y de varias versiones de *Comprar productos*, podríamos asignar algunos casos de uso al ciclo de desarrollo de la figura 7.2.

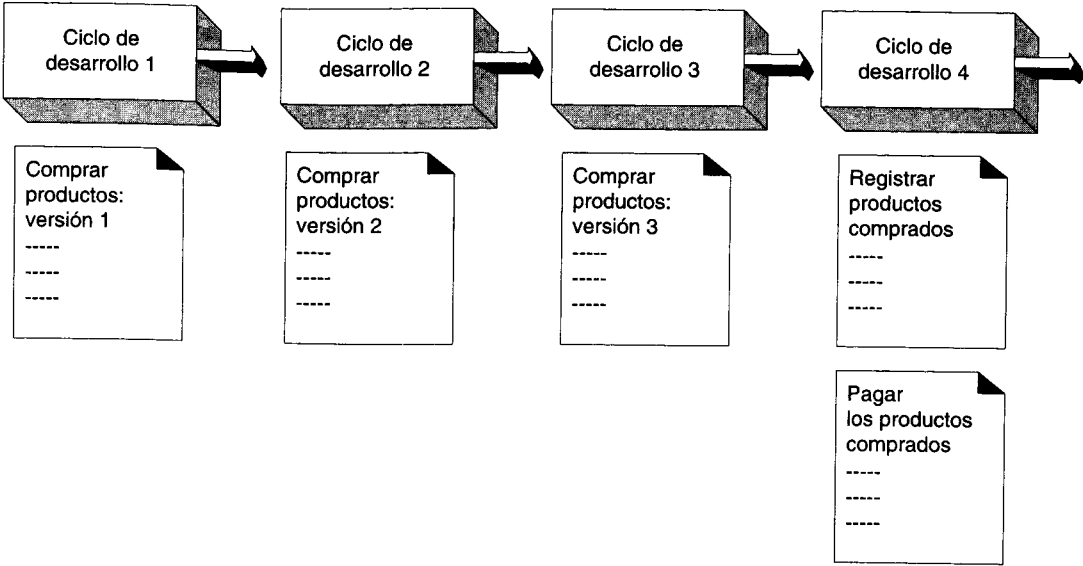


Figura 7.2 Asignación de los casos de uso a los ciclos de desarrollo.

- No se controla la caja de efectivo.
- En el recibo aparecen el nombre y la dirección de la tienda, la fecha y la hora de la venta.
- Ni la identificación del cajero ni la de TPDV aparecen en el recibo.
- Las ventas realizadas se registran en un documento histórico.

Caso de uso:	Comprar productos, versión 1
Actores:	Cliente (iniciador), Cajero.
Propósito:	Capturar una venta y su pago en efectivo.
Resumen:	Un Cliente llega a la caja con productos que desea comprar. El Cajero registra los productos comprados y recibe el pago en efectivo. Al terminar la transacción, el Cliente se marcha con los productos.
Tipo:	Primario y esencial.
Referencias cruzadas:	Funciones: R1.1, R1.2, R1.3, R1.5, R1.7, R1.9, R2.1.

Curso normal de los eventos

Acción de los actores	Respuesta del sistema
1. Este caso de uso comienza cuando un Cliente llega a un caja de TPDV con productos que desea comprar.	
2. El Cajero registra el código universal de producto (CUP) en cada producto. Si un producto se repite, el Cajero también puede introducir la cantidad.	3. Determina el precio del producto y agrega la información correspondiente a la transacción actual. Presenta la descripción y el precio del producto en cuestión.
4. Al terminar de capturar los productos, el Cajero indica a la TPDV que ya concluyó la captura.	5. Calcula el total de la venta y se lo presenta al Cliente.
6. El Cajero le indica al Cliente el total.	
7. El Cliente da un pago en efectivo —el efectivo “ofrecido”—, posiblemente mayor que el total de la venta.	

7.6 Versiones del caso de uso “Comprar productos”

Una vez que se ha decidido simplificar el caso y expresarlo, hay que escribir versiones cada vez más complejas. También hay que estipular las simplificaciones, las metas y las suposiciones de cada versión. Las siguientes secciones ofrecen sugerencias valiosas al respecto.

7.6.1 Versión 1 de Comprar productos

Simplificaciones, metas y suposiciones

- Pagos en efectivo exclusivamente.
- Sin mantenimiento de inventario.
- Es una tienda independiente, que no forma parte de una organización más grande.
- Captura manual del código universal de producto (CUP); sin lector de código de barras.
- No se calculan los impuestos.
- Sin cupones.
- El cajero no tiene que registrar las ventas; no se controla el acceso.
- No se lleva un registro de los clientes individuales ni de sus hábitos de compra.

Curso normal de los eventos

Acción de los actores	Respuesta del sistema
8. El Cajero registra el efectivo recibido.	9. Muestra al Cliente la diferencia. Genera un recibo.
10. El Cajero deposita el efectivo recibido y extrae la diferencia. El Cajero entrega al cliente el cambio y el recibo impreso.	11. Registra la venta terminada.
12. El Cliente se marcha con los productos comprados.	

7.6.2 Comprar productos: versión 2

Simplificaciones, metas y suposiciones

Las simplificaciones de la versión 1 se aplican también en esta versión, salvo que el pago puede efectuarse en efectivo, con tarjeta de crédito o con cheque. Las dos segundas formas de pago requieren autorización.

Caso de uso:	Comprar productos, versión 2
Actores:	Cliente (iniciador), Cajero.
Propósito:	Capturar una venta y su pago.
Resumen:	Un Cliente llega a la caja con productos que desea comprar. El Cajero registra los productos comprados y recibe un pago, que debe recibir autorización. Al terminar la transacción, el Cliente se marcha con los productos comprados, y así sucesivamente.

7.7 Resumen

Ya estamos preparados para hacer la transición a la fase del desarrollo iterativo. En el primer ciclo nos serviremos del caso de uso *Comprar productos: versión 1*.

INICIO DE UN CICLO DE DESARROLLO

Objetivos

- Resumir la transición de la fase de planeación y elaboración a la de construcción iterativa.

8.1 Inicio de un ciclo de desarrollo

Suponga que la fase de planeación y elaboración ha concluido y que los casos de uso han sido identificados, clasificados y programados, por lo menos en la primera pareja de ciclos. Se presenta entonces una transición muy importante: comienza la fase de construcción en la cual se cumplen los ciclos del desarrollo iterativo (figura 8.1). En el primer ciclo se ha decidido examinar una versión simplificada de *Comprar productos* que incluye solamente los pagos en efectivo y no el control de inventario.

Las actividades iniciales del ciclo se relacionan con la administración del proyecto. En el caso general, viene después (o, más probablemente, ocurre en paralelo) una sincronización de la documentación (por ejemplo, los diagramas) a partir del último ciclo con el estado real del código, porque los artefactos de diseño y los códigos difieren invariablemente durante la fase de codificación del último ciclo.

Entonces se inicia la fase de analizar (o de análisis), en la cual se investigan a fondo los problemas del ciclo actual. En esta fase, una de las primeras actividades consiste en desarrollar un modelo conceptual, tema que trataremos en el siguiente capítulo.

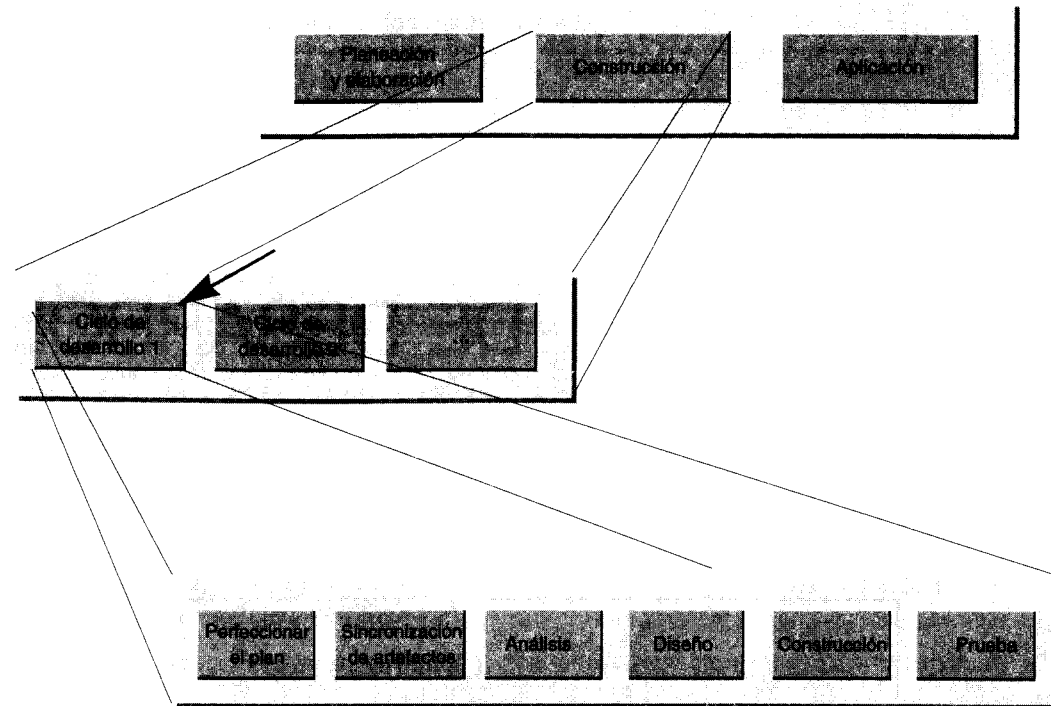


Figura 8.1 Un ciclo de desarrollo.

PARTE III FASE DE ANÁLISIS (1)

UNIVERSIDAD DE LA REPÚBLICA
FACULTAD DE INGENIERÍA
DEPARTAMENTO DE
DOCUMENTACIÓN Y BIBLIOTECA
MONTEVIDEO - URUGUAY

CONSTRUCCIÓN DE UN MODELO CONCEPTUAL

Objetivos

- Identificar los conceptos relacionados con los requerimientos del ciclo actual de desarrollo.
- Crear un modelo conceptual preliminar.
- Distinguir entre los atributos correctos y los incorrectos.
- Incorporar conceptos de *especificación* cuando convenga.
- Comparar los términos: concepto, tipo, interfaz y clase.

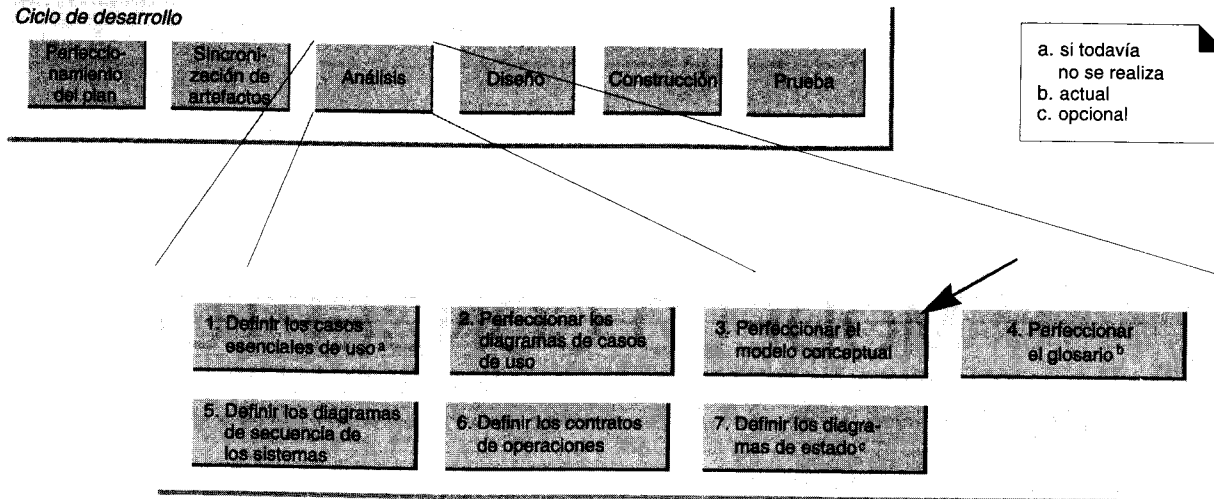
UNIVERSIDAD DE LA REPUBLICA
FACULTAD DE INGENIERIA
DEPARTAMENTO DE
DOCUMENTACION Y BIBLIOTECA
MONTEVIDEO - URUGUAY

9.1 Introducción

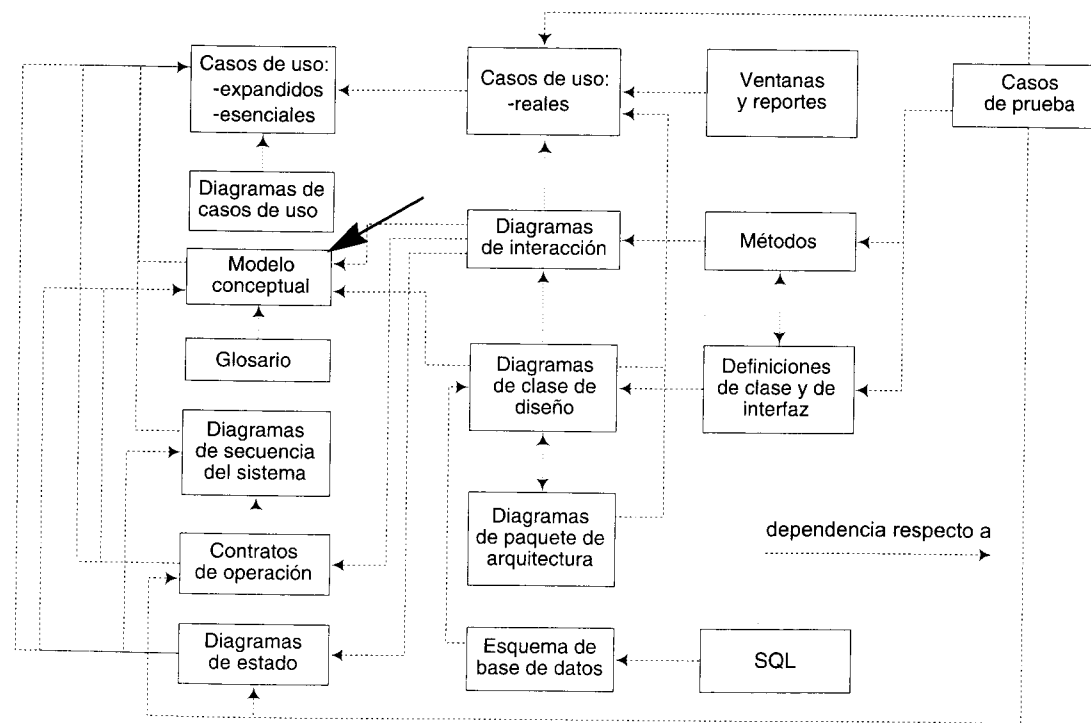
Un modelo conceptual explica (a sus creadores) los conceptos significativos en un dominio del problema; es el artefacto más importante a crear durante el análisis orientado a objetos.¹ En este capítulo estudiaremos los conocimientos preliminares en la creación de modelos conceptuales. En los dos siguientes examinaremos más detenidamente las habilidades relacionadas con la construcción de modelos conceptuales: observar atentamente los atributos y las asociaciones.

¹ Los casos de uso son un importante artefacto del análisis de requerimientos, pero realmente no están orientados a *objetos*. Ponen de relieve la vista del dominio a partir de un proceso.

Ciclo de desarrollo



Actividades de la fase de análisis dentro de un ciclo de desarrollo.



Dependencias de los artefactos durante la fase de construcción.

Identificar muchos objetos o conceptos constituye la esencia del análisis orientado a objetos, y el esfuerzo se compensa con los resultados conseguidos durante la fase de diseño e implementación.

La identificación de conceptos forma parte de una investigación del dominio del problema. El lenguaje UML contiene la notación en diagramas de estructura estática que explican gráficamente los modelos conceptuales.

Una cualidad esencial que debe ofrecer un modelo conceptual es que representa cosas del mundo real, no componentes del software.

9.2 Actividades y dependencias

Una de las primeras actividades centrales de un ciclo de desarrollo consiste en crear un modelo conceptual para los casos de uso del ciclo actual. Esto no puede hacerse si no se cuentan con los casos y con otros documentos que permitan identificar los conceptos (objetos). La creación no siempre es lineal; por ejemplo, el modelo conceptual puede formularse en paralelo con el desarrollo de los casos.

9.3 Modelos conceptuales

El paso esencial de un análisis o investigación orientados a *objetos* es descomponer el problema en conceptos u objetos individuales: las cosas que sabemos. Un **modelo conceptual** es una representación de conceptos en un dominio del problema [MO95, Fowler96]. En el UML, lo ilustramos con un grupo de **diagramas de estructura estática** donde no se define ninguna operación. La designación de *modelo conceptual* ofrece la ventaja de subrayar fuertemente una concentración en los conceptos del dominio, no en las entidades del software.

Puede mostrarnos:

- conceptos
- asociaciones entre conceptos
- atributos de conceptos

Por ejemplo, en la figura 9.1 vemos un modelo conceptual parcial del dominio de la tienda y las ventas. Explica gráficamente que el concepto de *Pago* y *Venta* son importantes en este dominio del problema, que *Pago* se relaciona con *Venta* en una forma que conviene señalar y que *Venta* tiene fecha y hora. Por ahora no son importantes para nosotros los detalles de la notación.

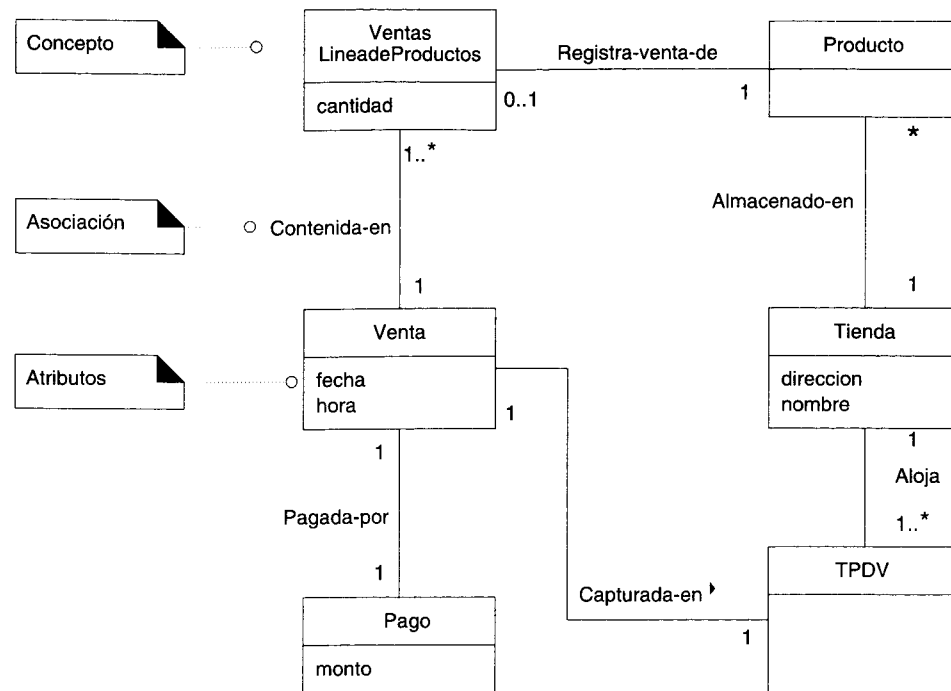


Figura 9.1 Modelo conceptual parcial. Los números en los extremos de la línea indica multiplicidad, la cual se describe en un capítulo subsecuente.

9.3.1 Conocimiento de la nomenclatura del dominio

Además de descomponer el espacio del problema en unidades comprensibles (conceptos), la creación de un modelo conceptual contribuye a esclarecer la terminología o nomenclatura del dominio. Podemos verlo como un modelo que comunica (a los interesados como pueden serlo los desarrolladores) cuáles son los términos importantes y cómo se relacionan entre sí.

9.3.2 Los modelos conceptuales no son modelos de diseño de software

Un modelo conceptual, como se advierte en la figura 9.2, es una descripción del dominio de un problema real, *no* es una descripción del diseño del software, como una clase de Java o de C++ (figura 9.3). Por ello los siguientes elementos no son adecuados en él:

- Los artefactos de software, como una ventana o una base de datos, salvo que el dominio a modelar se refiera a conceptos de software; por ejemplo, un modelo de interfaces gráficas para el usuario.

- Las responsabilidades o métodos.¹



Figura 9.2 Un modelo conceptual muestra conceptos del mundo real.

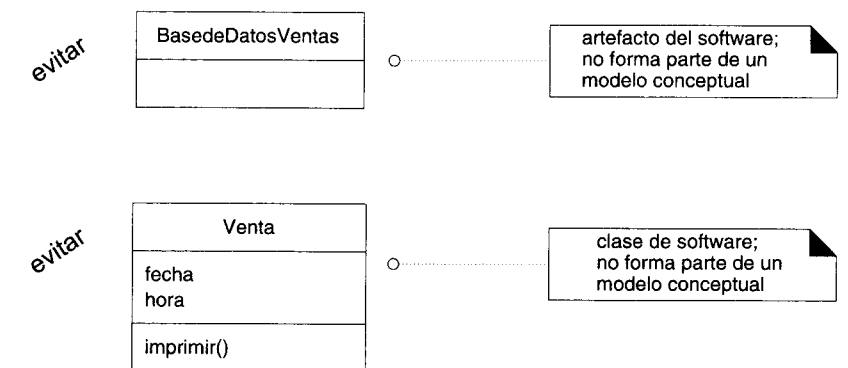


Figura 9.3 Un modelo conceptual no muestra los artefactos o clases del software.

9.3.3 Conceptos

En términos informales el concepto es una idea, cosa u objeto. En un lenguaje más formal, podemos considerarlo a partir de su símbolo, intensión² y extensión [MO95] (figura 9.4).

- **Símbolo:** palabras o imágenes que representan un concepto.
- **Intensión:** la definición del concepto.
- **Extensión:** el conjunto de ejemplos a que se aplica el concepto.

Consideremos, por ejemplo, el concepto del evento de una transacción de compra. Podemos optar por designarlo con el símbolo *Venta*. La intensión de una *Venta* puede afirmar que “representa el evento de una transacción de compra y tiene fecha y hora”. La extensión de *Venta* son todos los ejemplos de venta; en otras palabras, el conjunto de todas las ventas.

¹ Las responsabilidades normalmente se relacionan con entidades del software y los métodos siempre lo hacen; pero el modelo conceptual describe conceptos reales, no entidades del software. Durante la fase de *diseño* es muy importante tener en cuenta las responsabilidades; ya que no sólo forman parte de este modelo.

² Intensión: en oposición a extensión, designa el grado de una cualidad.

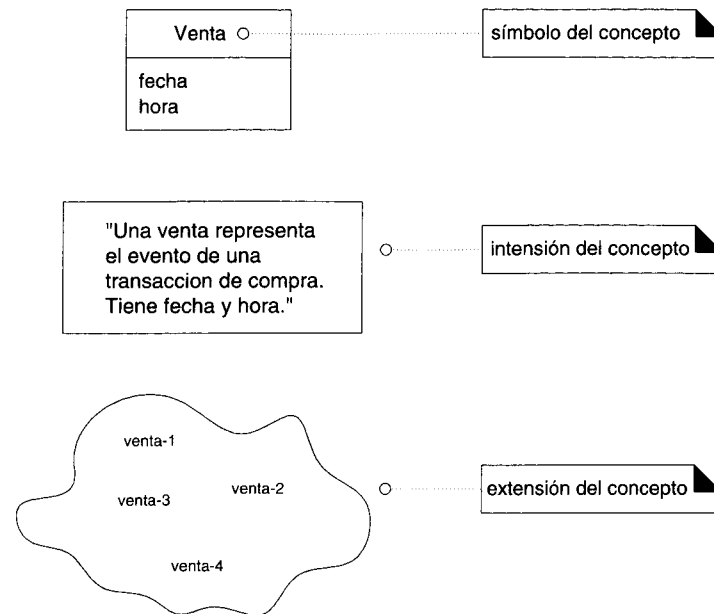


Figura 9.4 El concepto tiene un símbolo, intención y extensión.

Cuando se crea un modelo conceptual, por lo regular la vista del símbolo y de la intención de un concepto es el aspecto de mayor interés práctico.

9.3.4 Los modelos conceptuales y la descomposición

Los problemas de software a veces son complejos; la descomposición —divide y vencerás— es una estrategia que suele utilizarse para resolver la complejidad dividiendo el espacio del problema en unidades comprensibles. En el **análisis estructurado** la dimensión de la descomposición se realiza mediante procesos o *funciones*. En cambio, en el análisis orientado a objetos, se lleva a cabo fundamentalmente con conceptos.

Una distinción fundamental entre el análisis orientado a objetos y el análisis estructurado: división por conceptos (objetos) y no por funciones.

Por tanto, una tarea primordial de la fase de análisis consiste en identificar varios conceptos en el dominio del problema y documentar los resultados en un modelo conceptual.

9.3.5 Conceptos en el dominio del punto de venta

Por ejemplo, en el dominio del problema real de comprar productos en una tienda en una terminal de punto de venta (TPDV) intervienen los conceptos de *Tienda*, *TPDV* y una *Venta*. Por tanto, nuestro modelo conceptual (figura 9.5) puede incluir una *Tienda*, *TPDV* y una *Venta*.

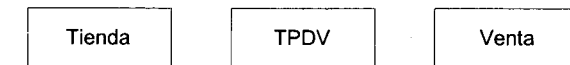


Figura 9.5 Modelo conceptual parcial en el dominio de la tienda.

9.4 Estrategias para identificar los conceptos

Nuestra meta es crear un modelo conceptual de conceptos interesantes o significativos del dominio en cuestión. En este caso, ello significa conceptos relacionados con el caso de uso *Comprar productos, versión 1*. La tarea fundamental será, pues, identificar los conceptos; se proponen dos estrategias.

La siguiente es una directriz de gran utilidad en la identificación de conceptos:

Es mejor exagerar y especificar un modelo conceptual con muchos conceptos refinados que no especificarlo cabalmente.

No piense que un modelo conceptual es más adecuado si tiene menos conceptos; generalmente suele suceder lo contrario.

Es frecuente omitir conceptos durante la fase inicial de identificación y descubrirlos más tarde cuando se examinen los atributos o asociaciones o durante la fase de diseño. Cuando se detecten, habrá que incorporarlos al modelo conceptual.

No se excluya un concepto simplemente porque los requerimientos no indiquen una necesidad evidente que permita recordar la información acerca de ella (criterio común de la construcción de modelos de datos para diseñar una base de datos relacional, pero no pertinente a la creación de modelos conceptuales) o porque el concepto carezca de atributos. Es perfectamente válido tener conceptos sin atributos o conceptos con un papel puramente de comportamientos en el dominio en vez de un papel informacional.

9.4.1 Obtención de conceptos a partir de una lista de categorías de conceptos

La creación de un modelo conceptual se comienza preparando una lista de conceptos idóneos a partir de la siguiente lista. Contiene muchas categorías comunes que vale la pena tener en cuenta, sin que importe el orden de importancia. Los ejemplos se tomaron de los dominios de la tienda y de las reservaciones de líneas aéreas.

Categoría del concepto	Ejemplos
objetos físicos o tangibles	<i>TPDV</i> <i>Avion</i>
especificaciones, diseño o descripciones de cosas	<i>EspecificaciondeProducto</i> <i>DescripciondeVuelo</i>
lugares	<i>Tienda</i> <i>Aeropuerto</i>
transacciones	<i>Venta, Pago</i> <i>Reservacion</i>
línea o renglón de elemento de transacciones	<i>VentasLineadeProducto</i>
papel de las personas	<i>Cajero</i> <i>Piloto</i>
contenedores de otras cosas	<i>Tienda, Cesto</i> <i>Avion</i>
cosas dentro de un contenedor	<i>Producto</i> <i>Pasajero</i>
otros sistemas de cómputo o electromecánicos externos al sistema	<i>SistemadeAutorizaciondeTarjetadeCredito</i> <i>ControldeTraficoAereo</i>
conceptos de nombres abstractos	<i>Hambre</i> <i>Acrofobia</i>
organizaciones	<i>DepartamentodeVentas</i> <i>ObjetoLineaAerea</i>
eventos	<i>Venta, Robo, Junta</i> <i>Vuelo, Accidente, Aterrizaje</i>

Categoría del concepto	Ejemplos
procesos (a menudo <i>no</i> están representados como conceptos, pero pueden estarlo)	<i>VentaUnProducto</i> <i>ReservacionAsiento</i>
reglas y políticas	<i>PoliticaReembolso</i> <i>PoliticaCancelaciones</i>
catálogos	<i>CatalogodeProducto</i> <i>CatalogodePartes</i>
registros de finanzas, de trabajo, de contratos de asuntos legales	<i>Recibo, Mayor, ContratodeEmpleo</i> <i>BitacoradeMantenimiento</i>
instrumentos y servicios financieros	<i>LineadeCredito</i> <i>Existencia</i>
manuales, libros	<i>ManualdePersonal</i> <i>ManualdeReparaciones</i>

9.4.2 Obtención de conceptos a partir de la identificación de frases nominales

Otra técnica muy útil (por su simplicidad) propuesta en [Abbot83] consiste en identificar las frases nominales en las descripciones textuales del dominio de un problema y considerarlas conceptos o atributos idóneos.

Este método hay que utilizarlo con mucha prudencia; no es posible encontrar mecánicamente correspondencias entre sustantivo y concepto, y además las palabras del lenguaje natural son ambiguas.

Pese a ello, esta técnica es fuente de inspiración. Los casos expandidos de uso son una excelente descripción que puede conseguirse con este análisis. Por ejemplo, puede usarse el caso de uso *Comprar productos, versión 1*.

Acción de los actores

- 1. Este caso de uso comienza cuando un **Cliente** llega a una **caja de TPDV** con **productos** que desea comprar.
- 2. El **Cajero** registra el **código universal de productos** (CUP) en cada **producto**.

Si hay más de un **producto**, el **Cajero** puede introducir también la **cantidad**.

Respuesta del sistema

- 3. Determina el **precio del producto** y a la **transacción de ventas** le agrega la información sobre el producto.
- Se muestran la **descripción** y el **precio del producto** actual.

Algunas de las frases nominales anteriores son conceptos idóneos; algunas pueden ser atributos de conceptos. Por favor, consulte el lector la sección siguiente y el capítulo dedicado a los atributos: en ellos encontrará sugerencias para distinguirlos.

Una debilidad de este enfoque es la imprecisión del lenguaje natural; varias frases nominales pueden designar el mismo concepto o atributo, entre otras ambigüedades que pueden presentarse. Pese a ello, no dudamos en recomendar usarlo en combinación con el método de *Lista de categoría de conceptos*.

9.5 Conceptos idóneos para el dominio del punto de venta

A partir de la *Lista de categoría de conceptos* y del análisis de frases nominales generamos una lista de conceptos adecuados para incluirlos en la aplicación del punto de venta. La lista está sujeta a la restricción de los requerimientos y simplificaciones que se consideren en el momento: los casos simplificados de uso de *Comprar productos*, versión 1.

TPDV	EspecificaciondeProducto
Producto	VentasLineadeProductos
Tienda	Cajero
Venta	Cliente
Pago	Gerente
Catálogo de Productos	

9.5.1 Objetos del informe: ¿se incluye el recibo en el modelo?

El recibo es un registro de una venta y de un pago, así como un concepto relativamente prominente en el dominio de ventas; ¿debe, pues, mostrarse en el modelo? En seguida se mencionan algunos factores que han de tenerse presentes:

- El recibo es un informe de una venta. En general, no conviene incluirlo en un modelo conceptual, ya que toda su información proviene de otras fuentes. Éste es un buen motivo para excluirlo.
- El recibo cumple un papel especial respecto a las reglas de la empresa: al portador le confiere el derecho de devolver los productos adquiridos. Esta es una razón para incorporarlo al modelo.

El recibo se excluirá, porque las devoluciones de productos no se incluyen en este ciclo de desarrollo. Se justificará su inclusión durante el ciclo que aborde el caso *Devolver productos*.

9.5.2 El modelo conceptual del punto de venta (sólo conceptos)

La lista anterior de los nombres de conceptos puede representarse gráficamente (figura 9.6) en la notación del diagrama de estructura estática de UML, a fin de mostrar la génesis del modelo conceptual.



Figura 9.6 Modelo conceptual inicial del dominio del punto de venta.

En capítulos posteriores trataremos de los atributos y asociaciones del modelo conceptual.

9.6 Directrices para construir modelos conceptuales

9.6.1 Cómo construir un modelo conceptual

Aplique los siguientes pasos para crear un modelo conceptual:

Para construir un modelo conceptual:

1. Liste los conceptos idóneos usando la *Lista de categorías de conceptos* y la identificación de la frase nominal relacionadas con los requerimientos en cuestión.
2. Dibújelos en un modelo conceptual.
3. Incorpore las asociaciones necesarias para registrar las relaciones para las cuales debe reservar un espacio en la memoria (tema que se expondrá en un capítulo posterior).
4. Agregue los atributos necesarios para cumplir con las necesidades de información (tema que se tratará en un capítulo posterior).

9.6.2 La asignación de nombres y el modelado de las cosas: el cartógrafo

La estrategia del cartógrafo se aplica a los mapas y a los modelos conceptuales.

Prepare un modelo conceptual inspirándose en la metodología del cartógrafo:

- Utilice los nombres existentes en el territorio.
- Excluya las características irrelevantes.
- No agregue cosas que no existan.

El modelo conceptual es una especie de mapa de conceptos o cosas de un dominio. Este enfoque pone de relieve el papel analítico de un modelo conceptual y sugiere lo siguiente:

- Los cartógrafos se sirven de nombres del territorio —no cambian los nombres de ciudades en sus mapas. En el caso de un modelo conceptual, ello significa *utilizar el vocabulario del dominio cuando se asignan nombres a los conceptos y a los atributos*. Por ejemplo, al desarrollar el modelo de una biblioteca, al cliente se le designará con los nombres que utilice el personal: *Visitante*, *Lector* u otro semejante.

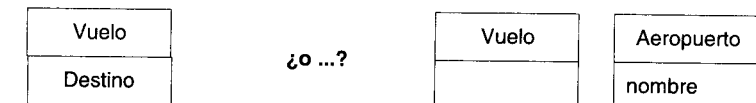
- Un cartógrafo elimina cosas en el mapa en caso de que no las juzgue pertinentes para el propósito que persigue; así, no es necesario que muestre la topografía ni las poblaciones. De modo análogo, un modelo conceptual puede excluir en el dominio del problema los conceptos que no se relacionen con los requerimientos. Por ejemplo, podemos omitir *Pluma* y *BolsadePapel* en nuestro modelo conceptual (con el conjunto actual de requerimientos), por no tener una función importante que sea obvia.
- Un cartógrafo no muestra cosas que no existan; por ejemplo, una montaña inexistente. En forma parecida, el modelo conceptual ha de excluir las cosas que *no* se encuentren en el dominio del problema en cuestión.

9.6.3 Un error que se comete frecuentemente al identificar los conceptos

Tal vez el error más frecuente cuando se crea un modelo conceptual es el de representar algo como atributo, cuando debió haber sido un concepto. Una regla práctica para no caer en él es:

Si en el mundo real no consideramos algún concepto X como número o texto, probablemente X sea un concepto y no un atributo.

Por ejemplo, pongamos el caso del dominio de las reservaciones en líneas aéreas. ¿Debería *Destino* ser un atributo de *Vuelo* o un concepto aparte *Aeropuerto*?



En el mundo real, un aeropuerto de destino no se considera número ni texto: es una cosa masiva que ocupa espacio. Por tanto, *Aeropuerto* debería ser un concepto.

En caso de duda, convierta el atributo en un concepto independiente.

9.7 Solución de los conceptos similares: comparación entre TPDV y Registro

Antaño, mucho antes del advenimiento de las terminales instaladas en el punto de venta, una tienda llevaba un *registro*: un libro donde asentaba las ventas y los pagos. Con el tiempo fue automatizado en un registro mecánico de efectivo. Hoy esa terminal desempeña el papel del registro (figura 9.7).

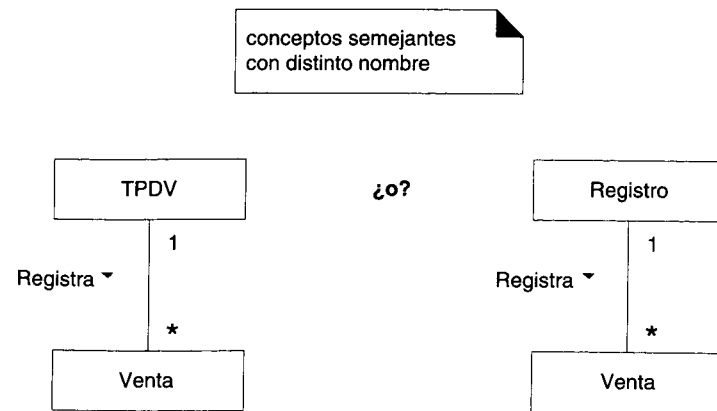


Figura 9.7 TPDV y registro son conceptos similares.

Un registro es una cosa que asienta las ventas y los pagos, pero también lo es la terminal instalada en el punto de ventas. No obstante, el término *registro* parece tener un significado más abstracto y denota una implementación menos orientada que *TPDV*. Pues bien, ¿en el modelo conceptual deberíamos utilizar el símbolo *Registro* en vez de *TPDV*?

Primero, una regla práctica consiste en que un modelo conceptual no es absolutamente correcto ni erróneo, sino de mayor o menor utilidad; es una herramienta de la comunicación.

Conforme al principio del cartógrafo, *TPDV* es un término usual en el territorio, por lo cual es un símbolo útil desde el punto de vista del conocimiento y la comunicación. *Registro* es atractivo y útil, según la meta de crear modelos que representen abstracciones y que no dependan de la implementación.¹

Ambas opciones tienen sus bondades. En este caso de estudio hemos escogido *TPDV* de modo un tanto arbitrario; también pudimos haber escogido *Registro*.

9.8 Construcción de un modelo del mundo *irreal*

Algunos sistemas de software están destinados a dominios que presentan muy poca semejanza con los dominios naturales o con los de las empresas; un ejemplo lo constituye el software de las telecomunicaciones. Todavía es posible crear un modelo conceptual en esos dominios, pero se requiere un alto grado de abstracción y abandonar los diseños comunes.

¹ Nótese que antaño un *registro* era simplemente una implementación posible de la manera de registrar las ventas. Con el tiempo, su significado ha ido generalizándose.

Enumeramos algunos conceptos adecuados que se relacionan con un intercambio de telecomunicaciones: *Mensaje, Conexión, Diálogo, Ruta, Protocolo*.

9.9 Especificación o descripción de conceptos

Suponga lo siguiente:

- La instancia de *Elemento* representa una entidad física de una tienda; puede ser incluso un número serial.
- Un *Elemento* tiene una descripción, precio y código universal de producto, los cuales no están registrados en ninguna otra parte.
- Todos los que trabajan en la tienda sufren amnesia.
- Cada vez que se vende un elemento físico, en "terreno del software" se elimina una instancia del software correspondiente a *Elemento*.

Con estas suposiciones, preguntamos: ¿qué sucede en el siguiente escenario?

Existe gran demanda de una nueva hamburguesa: ObjetoHamburguesa. La tienda vende todas sus existencias, lo cual significa que todas las instancias de *Elemento* de ObjetoHamburguesa se cancelan en la memoria de la computadora.

Ahora bien, ésta es la esencia del problema: si alguien pregunta: "¿Cuánto cuesta el ObjetoHamburguesa?", nadie podrá contestarle porque la memoria de su precio se anexó a las instancias inventariadas, que fueron eliminándose conforme se vendían.

Nótese asimismo que el modelo actual, si se implementa en el software tal como se describe, posee datos duplicados y maneja ineficientemente el espacio, porque la descripción, el precio y el código universal de producto se duplica en cada instancia de *Elemento* del mismo producto.

9.9.1 Necesidad de las especificaciones

El problema anterior demuestra la necesidad de un concepto de objetos que son especificaciones o descripciones de otras cosas. Para resolver el problema del *Elemento* lo que se necesita es una *EspecificaciondeProducto* (o *EspecificaciondeElemento*, *DescripciondeProducto*, ...) concepto que registra la información sobre los elementos. Una *EspecificaciondeProducto* no representa un *Elemento*, sino una descripción acerca de ellos. Nótese que, aunque todos los elementos inventariados se vendan y se eliminen sus instancias correspondiente de software, se conserva la *EspecificaciondeProducto*.

La descripción o especificación de objetos se relacionan bastante con aquella que describen. En un modelo conceptual, se acostumbra estipular que una *EspecificacionX* describe una *X*.

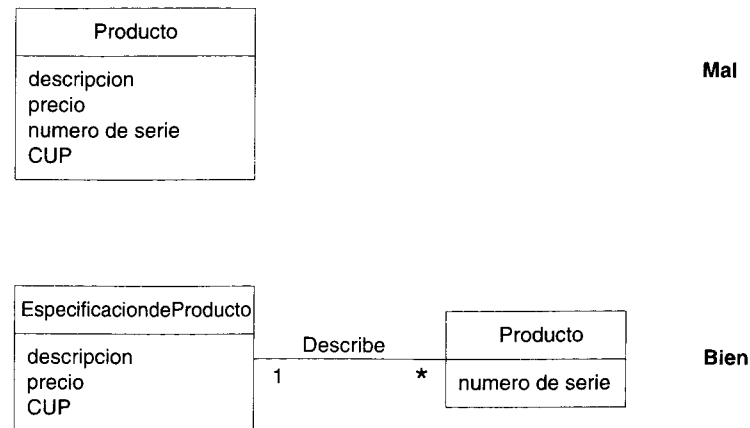


Figura 9.8 Especificaciones de otras cosas. El signo “*” significa una multiplicidad de “muchos”. Indica que una *EspecificacioneProducto* puede describir muchos (*) *Productos*.

La necesidad de especificar los conceptos es frecuente en los dominios de ventas y productos. También lo es en la manufactura, donde se requiere una *descripción* de lo manufacturado que se distingue de la cosa manufacturada. El tiempo y el espacio han sido incluidos al explicar la causa de la especificación de conceptos, por ser muy comunes; no se trata de un concepto poco usual de la construcción de modelos.

9.9.2 ¿Cuándo se requiere especificar los conceptos?

Las siguientes directrices indican cuándo emplear las especificaciones.

Incorpore una especificación o descripción de conceptos (por ejemplo, *EspecificacioneProducto*) cuando:

- La eliminación de las instancias de las cosas que describen *Elemento*, por ejemplo, da por resultado una pérdida de la información que ha de conservarse, debido a la asociación incorrecta de la información con lo eliminado.
- Reduce información redundante o duplicada.

9.9.3 Otro ejemplo de especificación

He aquí un último ejemplo: imagine que una compañía aérea sufre el accidente fatal de uno de sus aviones. Suponga que todos los vuelos quedan cancelados por seis meses, mientras se lleva a cabo la investigación. Suponga además que, cuando se cancelan los vuelos, sus correspondientes objetos de software *Vuelo* se eliminan en la memoria de la computadora. Así, todos los objetos *Vuelo* quedan eliminados tras el accidente.

Si el único registro del aeropuerto al cual se dirige un vuelo está en las instancias *Vuelo*, que representan vuelos específicos en determinado día y hora, ya no habrá un registro de qué rutas tiene la línea aérea.

Para resolver este problema, se requiere una *DescripcioneVuelo* (o *EspecificacioneVuelo*) que describa un vuelo y su ruta, aun cuando no esté programado un vuelo determinado (figura 9.9).

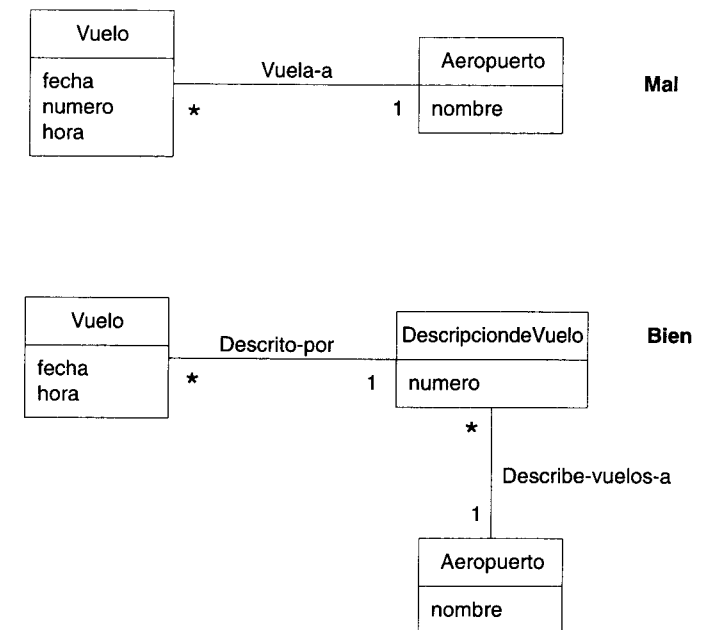


Figura 9.9 Especificaciones de otros elementos.

9.10 Definición de términos en el lenguaje UML

En UML, se emplean los términos “clase” y “tipo”, no así “concepto”. No existe consenso unánime respecto al significado de clase y tipo; así que para evitar ambigüedades el UML define rigurosamente ambos términos tal como se utilizan en su meta-modelo (el modelo de UML), aunque otros autores y profesionales usan definiciones alternas y antagónicas.

Cualquiera que sea la definición, lo importante es su utilidad para distinguir entre la perspectiva de un analista de dominios que observa conceptos reales, como venta, y los diseñadores de software que especifican entidades de programas; por ejemplo, la clase *Venta* en Java. El UML sirve, entre otras cosas, para explicar concretamente las perspectivas de notación y terminología muy afines; de ahí la importancia de tener presente qué perspectiva va a adoptarse (un análisis, un diseño o una vista de la implementación).

Para simplificar nuestra exposición, en este libro con el término “concepto” designaremos cosas del mundo real y con el de “clase”, las especificaciones e implementaciones del software.

La definición de **clase** en UML es “una descripción de un conjunto de objetos que comparten los mismos atributos, operaciones, métodos, relaciones y semántica” [BJR97]. Algunos autores limitan la definición de clase a una implementación concreta de software, digamos una clase en Java [Fowler96]. Pero, en el UML, este vocablo tiene una acepción más general: abarca especificaciones que anteceden a la implementación. En ese lenguaje, a una clase implementada de software se le llama más concretamente **clase de implementación**.

En UML, una **operación** es “un servicio que puede solicitarse a un objeto para que realice un comportamiento” [BJR97], y **método** es la implementación de una operación que especifica el algoritmo o procedimiento de esta última.

La definición de **tipo** en UML se asemeja a la de clase —describe un conjunto de objetos parecidos con atributos y operaciones—, pero no puede incluir métodos. De ello se deduce que un tipo es una *especificación* de una entidad de software y no una implementación. Ello significa también que un tipo de UML es independiente del lenguaje.

Aunque no sea estrictamente exacto dentro del contexto del lenguaje UML, este libro a veces utiliza los vocablos “concepto” y “tipo” indistintamente, porque en el uso coloquial el vocablo “tipo” a menudo se define como sinónimo de un concepto del mundo real [MO95].

El término **interfaz** se define como un conjunto de operaciones visibles en el exterior. En el UML, puede estar asociada a tipos y clases (y también a paquetes que agrupan elementos). Aunque los conceptos reales pueden tener una interfaz (la de un teléfono, por ejemplo), el término suele emplearse dentro del contexto de una interfaz para entidades de software, como la interfaz *Runnable* de Java.

9.11 Modelos Patrón

El Modelo Conceptual se compone de diagramas estáticos de estructura UML que ilustran conceptos en el dominio, como se muestra en la figura 9.10.

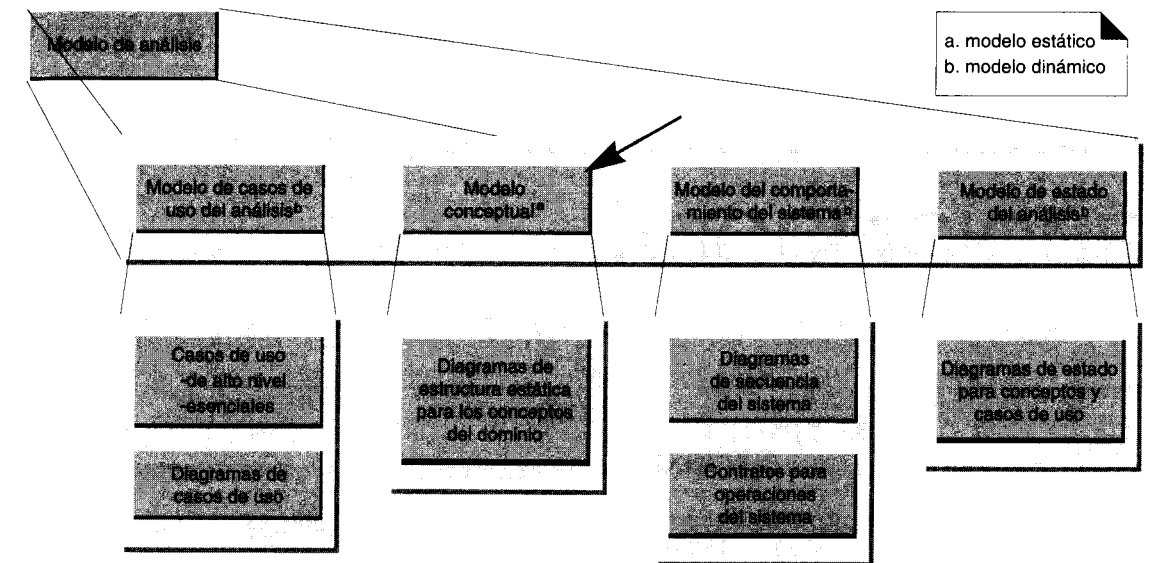


Figura 9.10 El modelo de análisis.

MODELO CONCEPTUAL: AGREGACIÓN DE LAS ASOCIACIONES

Objetivos

- Identificar las asociaciones de un modelo conceptual.
- Distinguir entre las asociaciones que es necesario conocer y las que se requieren sólo para la comprensión.

10.1 Introducción

Es necesario identificar las asociaciones de los conceptos que se requieren para satisfacer los requerimientos de información de los casos de uso en cuestión y los que contribuyen a entender el modelo conceptual. En el presente capítulo examinaremos la identificación de las asociaciones adecuadas y las incorporaremos al modelo conceptual del sistema del punto de venta.

10.2 Asociaciones

La **asociación** es una relación entre dos conceptos que indica alguna conexión significativa e interesante entre ellos (figura 10.1).

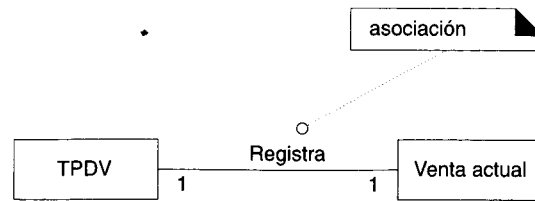


Figura 10.1 Asociaciones.

En el lenguaje UML se describen como “relaciones estructurales entre los objetos de diversos tipos”.

10.2.1 Criterios de las asociaciones útiles

Las asociaciones que vale la pena mencionar suelen incluir el conocimiento de una relación que ha de preservarse durante algún tiempo: puede tratarse de milisegundos o años según el contexto. En otras palabras, ¿entre qué objetos hemos de tener algún recuerdo de una relación? Por ejemplo, ¿debemos recordar cuáles instancias de *VentasLineadeProducto* están asociadas a la instancia *Venta*? Claro que sí, porque de lo contrario no sería posible reconstruir la venta, imprimir el recibo ni calcular el total de la venta.

Examine la conveniencia de incluir las siguientes asociaciones en un modelo conceptual:

- Las asociaciones en que el conocimiento de la relación ha de ser preservado durante algún tiempo (asociaciones que “deben conocerse”).
- Las asociaciones provenientes de la *Lista de asociaciones comunes*.

En cambio, ¿necesitamos recordar una relación entre una *Venta* actual y un *Gerente*? La respuesta es negativa: los requerimientos no indican que haga falta ese tipo de relación. Tal vez no sea incorrecto mostrar una relación entre una *Venta* y *Gerente*, pero no es indispensable ni útil dentro del contexto de nuestros requerimientos.

10.3 Notación de las asociaciones en el UML

Una asociación se representa como una línea entre conceptos con el nombre de la asociación. Ésta es intrínsecamente bidireccional, o sea es posible un nexo lógico entre los objetos de un tipo y los del otro. Este vínculo es totalmente abstracto; *no* es una afirmación sobre las conexiones entre las entidades del software.

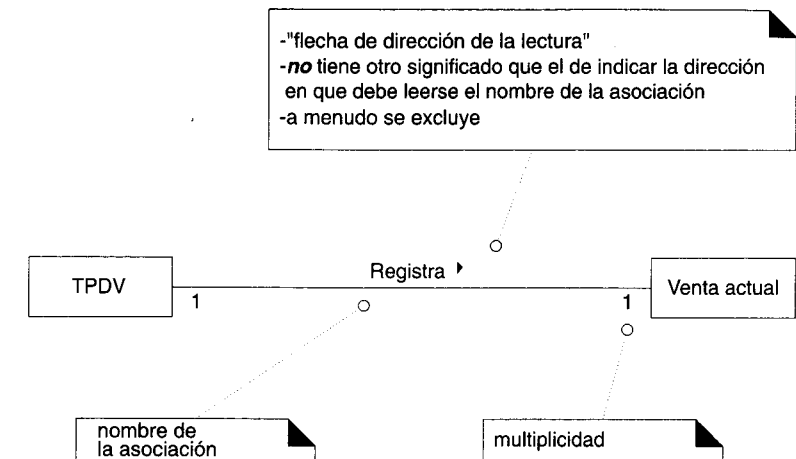


Figura 10.2 Notación de las asociaciones en el lenguaje UML.

Los extremos de una asociación pueden contener una expresión de multiplicidad que indique la relación numérica entre las instancias de los conceptos.

Una flecha opcional de la dirección de la lectura (o “flecha de la dirección del nombre”) indica la dirección en que debe leerse el nombre de la asociación; no denota la dirección de visibilidad o de navegación. En su ausencia, por convención la asociación se lee de izquierda a derecha o de arriba hacia abajo, aunque el UML no hace de esto una regla (figura 10.2).

La flecha de dirección de la lectura no tiene un valor semántico; tan sólo es una ayuda para leer el diagrama.

10.4 Identificación de las asociaciones: lista de asociaciones comunes

Comience a agregar las asociaciones utilizando la lista anexa. Contiene categorías comunes que normalmente vale la pena incluir. Los ejemplos están tomados de los dominios de la tienda y de la reservación de las líneas aéreas.

Categoría	Ejemplos
A es una parte física de B	Caja—TPDV Ala—Avion
A es una parte lógica de B	VentasLineadeProducto—Venta TramodeVuelo—RutadeVuelo
A está físicamente contenido en B	TPDV—Tienda, Producto—Estante Pasajero—Avion
A está contenido lógicamente en B	DescripciondeProducto—Catalogo Vuelo—ProgramadeVuelo
A es una descripción de B	DescripciondeProducto—Producto DescripciondeVuelo—Vuelo
A es un elemento de línea en una transacción o reporte B	VentasLineadeProducto—Venta TrabajodeMantenimiento—Mantenimiento
A se conoce/introduce/registra/presenta/captura en B	Venta—TPDV Reservacion—ListadePasajeros
A es miembro de B	Cajero—Tienda Piloto—Avion
A es una subunidad organizacional de B	Departamento—Tienda Mantenimiento—Linea Aerea
A usa o dirige a B	Cajero—TPDV Piloto—Avion
A se comunica con B	Cliente—Cajero AgentedeReservaciones—Pasajero
A se relaciona con una transacción B	Pago—Venta Pasajero—Boleto

Categoría	Ejemplos
A es una transacción relacionada con otra transacción B	Pago—Venta Reservacion—Cancelacion
A está contiguo a B	TPDV—TPDV Ciudad—Ciudad
A es propiedad de B	TPDV—Tienda Avion—Linea aerea

10.4.1 Asociaciones de alta prioridad

A continuación se enumeran algunas categorías de alta prioridad que siempre conviene incluir en un modelo conceptual:

- A es una *parte física o lógica* de B.
- A está *física o lógicamente contenido* en B.
- A está *registrado* en B.

10.5 ¿Qué grado de detalle deberían tener las asociaciones?

Las asociaciones son importantes, pero una falla habitual al crear modelos conceptuales es el excesivo tiempo que, durante la investigación, se dedica al intento de descubrir las asociaciones. Es indispensable convencerse de lo siguiente:

Es mucho más importante identificar los *conceptos* que las asociaciones. El tiempo consagrado a la creación del modelo conceptual debería destinarse a identificar los conceptos, no las asociaciones.

10.6 Directrices de las asociaciones

Directrices de las asociaciones:

- Concentrarse en las asociaciones en que el conocimiento de la relación ha de preservarse durante algún tiempo (asociaciones que "es necesario conocer").
- Es más importante identificar los *conceptos* que las asociaciones.
- Muchas asociaciones tienden a confundir el modelo conceptual en vez de aclararlo. A veces se requiere mucho tiempo para descubrirlas, y los beneficios son escasos.
- No incluir las asociaciones redundantes ni las derivables.

10.7 Papeles

A los extremos de una asociación se les llama **papeles**. Éstos pueden tener:

- nombre
- expresión de multiplicidad
- navegabilidad

En este momento se investiga la multiplicidad, pero las dos características restantes se estudiarán en capítulos posteriores.

10.7.1 Multiplicidad

La **multiplicidad** define cuántas instancias de un tipo *A* pueden asociarse a una instancia del tipo *B* en determinado momento (figura 10.3).

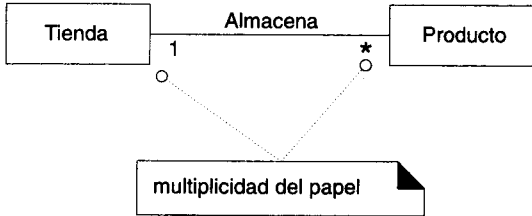


Figura 10.3 Multiplicidad en una asociación.

Por ejemplo, una instancia individual de una *Tienda* puede asociarse a "muchas" instancias (cero o más marcadas con *) de *Producto*. En la figura 10.4 se ofrecen algunos ejemplos de las expresiones de multiplicidad.

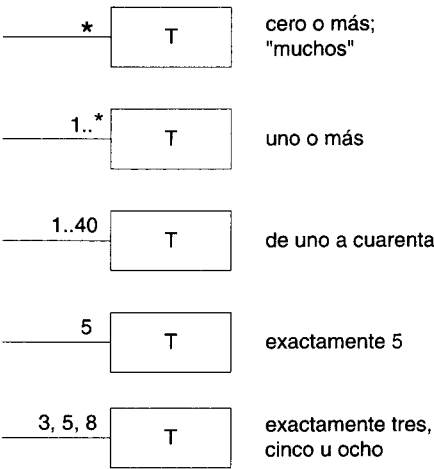


Figura 10.4 Valores de la multiplicidad.

En el UML, el valor de multiplicidad depende del contexto. Rumbaugh [Rumbaugh91] da un excelente ejemplo de *Persona* y *Compañía* en la asociación *Trabaja-para*. Del contexto del modelo dependerá indicar si una instancia de *Persona* se aplica a una o a varias instancias de *Compañía*; al departamento de impuestos le interesan *muchas*; a un sindicato probablemente le interese sólo *una*.

10.8 Asignación de nombre a las asociaciones

Se asigna nombre a una asociación basándose en el formato *Nombre de Tipo-Frase Nominal-Nombre de Tipo*, donde la frase nominal genera una secuencia que es legible y significativa dentro del contexto del modelo.

Los nombres de las asociaciones comienzan con una mayúscula. Una frase nominal debe construirse con guiones. En la figura 10.5, la dirección por omisión en que debe leerse el nombre de una asociación es de izquierda a derecha o de arriba hacia abajo. No es así en la dirección por omisión de UML, aunque se trata de una convención relativamente usual.

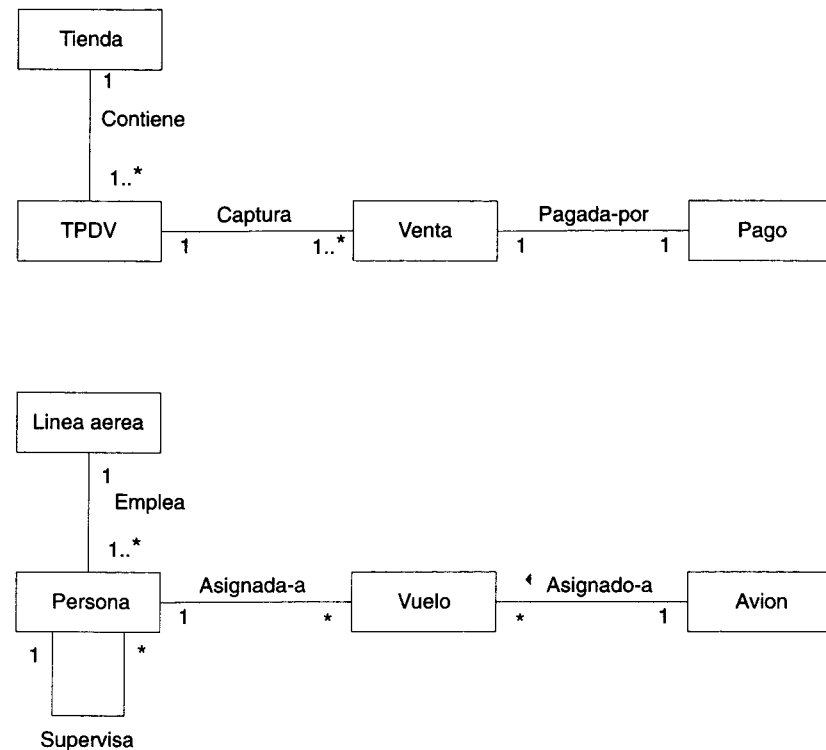


Figura 10.5 Nombres de las asociaciones.

10.9 Asociaciones múltiples entre dos tipos

Dos tipos pueden tener varias asociaciones entre ellos; esto sucede con frecuencia. No hay un ejemplo sobresaliente en nuestro sistema TPDV, pero en el dominio de la línea aérea encontramos uno en las relaciones entre *Vuelo* y *Aeropuerto* (figura 10.6). Las asociaciones volar-hacia y volar-de son relaciones netamente diferentes que han de mostrarse por separado. Adviértase asimismo que no se garantiza que todos los vuelos aterricen en un aeropuerto!

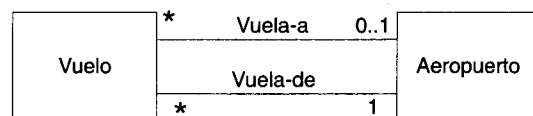


Figura 10.6 Asociaciones múltiples.

10.10 Asociaciones e implementación

Durante la fase de análisis, una asociación *no* es una proposición sobre los flujos de datos, variables de instancia ni conexiones de objetos en una solución de software; es una proposición de que una relación es significativa en un sentido puramente analítico: en el mundo real. En la práctica, muchas de estas relaciones suelen implementarse en los programas como trayectorias de navegación y visibilidad; pero su presencia en una vista investigadora o analítica de un modo conceptual no requiere la implementación.

Cuando se crea un modelo conceptual, podemos definir asociaciones que no se necesitarán en la construcción. Y a la inversa: podemos descubrir asociaciones que han de ser implementadas pero que se omitieron durante la fase de análisis. En tal caso, habría que actualizar el modelo para que incluyan esos descubrimientos.

Más adelante explicaremos las formas de implementar las asociaciones en un lenguaje de programación orientado a objetos (lo usual es servirse de un atributo que apunte a una instancia de la clase asociada), pero por ahora conviene verlas como meras expresiones analíticas, *no* como proposiciones acerca de una solución de base de datos ni de software. Como de costumbre, posponemos las consideraciones de diseño y así nos liberamos de información y decisiones prematuras en el modelo de análisis, a la vez que aumentamos al máximo nuestras opciones para después.

10.11 Asociaciones del dominio del punto de venta

Ahora podemos agregar asociaciones a nuestro modelo conceptual del sistema del punto de venta. Deberíamos incorporar las que indican los requerimientos (los casos de uso, por ejemplo), las que conllevan la necesidad de recordar o que de alguna otra manera nos sugiere nuestra percepción del dominio del problema. Cuando acometamos un nuevo problema, hay que repasar y estudiar las categorías comunes de las asociaciones mencionadas en páginas anteriores, pues representan muchas de las asociaciones pertinentes que generalmente es preciso registrar.

10.11.1 Relaciones inolvidables en la tienda

La siguiente muestra de asociaciones se justifica por la necesidad de conocerlas. Se funda en los casos de uso que hemos venido examinando.

TPDV Captura Venta

Para conocer la venta actual genera un total, e imprime un recibo.

<i>Venta pagada en efectivo</i>	Para saber si se pagó la venta, relaciona la cantidad ofrecida con el total de la venta e imprime un recibo.
<i>CatalogodeProductos registra EspecificaciondeProducto</i>	Para recuperar una <i>EspecificaciondeProducto</i> , con un código universal de producto.

10.11.2 Aplicación de la categoría de la lista de comprobación de las asociaciones

Recorreremos la lista de comprobación, basándonos en los tipos anteriormente identificados y teniendo presentes los requerimientos actuales del caso de uso.

Categoría	Sistema TPDV
A es una parte física de B	<i>no aplicable</i>
A es una parte lógica de B	<i>VentasLineadeProducto—Venta</i>
A está contenido físicamente en B	<i>TPDV—Tienda</i> <i>Producto—Tienda</i>
A está contenido lógicamente en B	<i>EspecificaciondeProducto—CatalogodeProductos</i> <i>CatalogodeProductos—Tienda</i>
A es un descripción de B	<i>EspecificaciondeProducto—Producto</i>
A es una línea de una transacción o reporte B	<i>VentasLineadeProducto—Venta</i>
A se introduce/registra/presenta/captura en B	<i>Ventas(terminadas)—Tienda</i> <i>Venta(actual)—TPDV</i>
A es miembro de B	<i>Cajero—Tienda</i>
A es una subunidad organizacional de B	<i>No aplicable</i>
A usa o dirige a B	<i>Cajero—TPDV</i> <i>Gerente—TPDV</i> <i>Gerente—Cajero, pero probablemente no aplicable.</i>

Categoría	Sistema TPDV
A se comunica con B	<i>Cliente—Cajero</i>
A se relaciona con una transacción B	<i>Cliente—Pago</i> <i>Cajero—Pago</i>
A es una transacción relacionada con otra transacción B	<i>Pago—Venta</i>
A sigue a B	<i>TPDV—TPDV, pero probablemente no aplicable</i>
A es propiedad de B	<i>TPDV—Tienda</i>

10.12 Modelo conceptual del punto de venta

El modelo conceptual de la figura 10.7 muestra un conjunto de conceptos y asociaciones idóneos para nuestra aplicación al punto de venta. Las asociaciones se tomaron principalmente de la lista de comprobación de asociaciones adecuada.

10.12.1 ¿Se conservan sólo las asociaciones que deben conocerse?

El conjunto de asociaciones que se incluye en el modelo conceptual de la figura 10.7 se obtuvo de manera bastante mecánica a partir de la lista de comprobación. Pero tal vez hay que ser más selectivos con las asociaciones que contendrá nuestro modelo conceptual. Desde el punto de vista de la comunicación, no conviene saturar el modelo con asociaciones que no sean indispensables y que no mejoren nuestro conocimiento. El exceso de ellas no aclara sino que oscurece la situación.

Como señalamos con anterioridad, recomendamos los siguientes criterios para presentar las asociaciones:

- Concentrarse en las asociaciones en que el conocimiento de la relación ha de ser preservado durante algún tiempo (asociaciones “que deben conocerse”).
- No incluir las asociaciones redundantes ni las derivables.

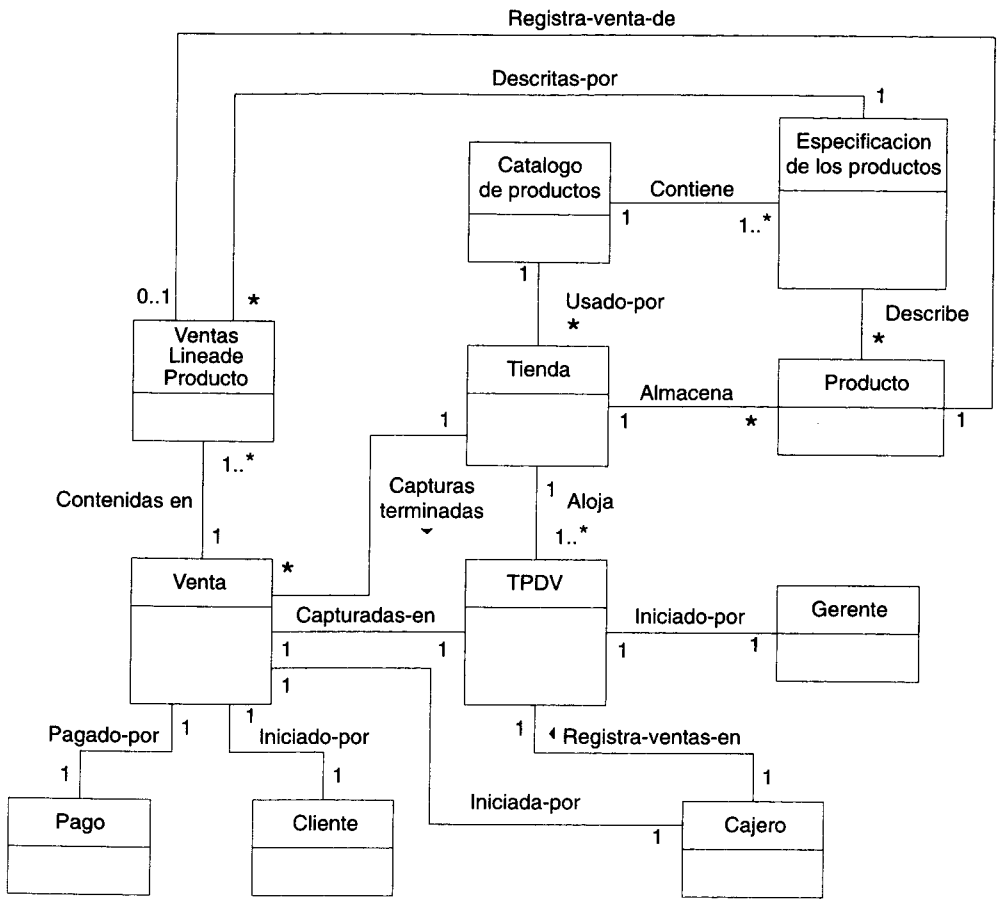


Figura 10.7 Modelo conceptual aplicado al punto de venta.

Conforme a nuestra recomendación, no son indispensables todas las asociaciones que se muestran en cierto momento. Estudie detenidamente la tabla anexa:

Asociación	Explicación
Venta capturada-por Cajero	Los requerimientos no indican la necesidad de conocer ni de registrar al cajero actual. Además, es derivable si existe la asociación TPDV Usado-por Cajero.
TPDV Usado-por Cajero	Los requerimientos no indican la necesidad de conocer ni de registrar al cajero actual.

Asociación	Explicación
TPDV Iniciado-por Gerente	Los requerimientos no indican la necesidad de conocer ni de registrar al gerente que inició un TPDV.
Venta Iniciada-por Cliente	Los requerimientos no indican la necesidad de conocer ni de registrar al cliente actual que inició una venta.
Tienda Almacena Producto	Los requerimientos no indican la necesidad de conocer o mantener la información del inventario.
VentasLineadeProducto Registra-venta-de-Producto	Los requerimientos no indican la necesidad de mantener la información de inventario.

Nótese que la capacidad de justificar una asociación atendiendo a la necesidad de conocerla depende de los requerimientos; un cambio de ellos —por ejemplo, exigir que la identificación del cajero aparezca en un recibo— altera la necesidad de recordar una relación.

A partir del análisis anterior, *tal vez* se justifique suprimir las asociaciones en cuestión.

10.12.2 Las asociaciones que necesitan conocerse y las que facilitan la comprensión

Un criterio estricto de la necesidad de conocer aplicado a la conservación de asociaciones dará origen a un “modelo mínimo de información” sobre lo que se requiere para construir un modelo del dominio del problema, modelo que estará acotado por los requerimientos en cuestión. Pero este procedimiento puede producir un modelo que no facilite (ni a los diseñadores ni a terceros) la comprensión cabal del dominio.

Algunas veces vemos el modelo conceptual no como un modelo riguroso de información, sino como una herramienta de la comunicación, con la cual intentamos entender los conceptos importantes y sus relaciones. Desde esta perspectiva, eliminar algunas asociaciones que no reclamen estrictamente el criterio de la necesidad de conocer puede originar un modelo inadecuado: no comunica las ideas ni las relaciones más importantes.

Esto lo vemos, por ejemplo, en la aplicación al punto de venta: aunque conforme al criterio estricto de la necesidad de conocer, quizá no sea preciso registrar *Venta Iniciada-por Cliente*, su ausencia omite un aspecto importante para comprender el dominio: el hecho de que un cliente genera ventas.

En lo tocante a las asociaciones, un buen modelo ocupa un punto intermedio de un modelo basado en la necesidad mínima de conocimiento y otro que contenga todas las relaciones posibles. ¿Cuál es el criterio básico para juzgar su valor? Uno podría ser:

¿cumple con todos los requerimientos de la necesidad de conocer y además comunica claramente una comprensión esencial de los conceptos importantes en el dominio del problema?

Enfatice las asociaciones que deben conocerse, pero incorpore también las opcionales que se requieren sólo para la comprensión, con el fin de enriquecer el conocimiento básico del dominio.

MODELO CONCEPTUAL: AGREGACIÓN DE LOS ATRIBUTOS

Objetivos

- Identificar los atributos en un modelo conceptual.
- Distinguir entre atributos correctos e incorrectos.

11.1 Introducción

Es necesario identificar los atributos de los conceptos que se necesitan para satisfacer los requerimientos de información de los casos de uso en cuestión. En este capítulo examinaremos la identificación de los atributos idóneos y le agregaremos atributos al modelo conceptual del dominio, que hemos aplicado al punto de venta.

Recuérdese que el modelo conceptual es una representación de cosas reales, no de componentes del software. Cualquier afirmación concerniente a los atributos ha de interpretarse dentro del contexto de entidades del mundo real.

11.2 Atributos

Un **atributo** es un valor lógico de un dato de un objeto.

Incluya los siguientes atributos en un modelo conceptual:
Aquellos en que los requerimientos (por ejemplo, los casos de uso) indican o conllevan la necesidad de recordar información.

Por ejemplo, un recibo de ventas normalmente incluye la fecha y la hora. En consecuencia, el concepto *Venta* requiere los atributos *fecha* y *hora*.

11.3 Notación de los atributos en el UML

Los atributos se muestran en la segunda sección de la sección de conceptos (Figura 11.1). Es opcional indicar su tipo.

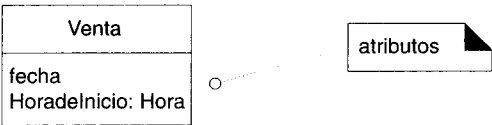


Figura 11.1 Concepto y atributos.

11.4 Tipos de atributos válidos

Hay algunas cosas que no deberían representarse como atributos, sino más bien como asociaciones. En esta sección trataremos de los atributos válidos.

11.4.1 Conserve simples los atributos

Los tipos más simples de atributos son los que, en la práctica, suelen considerarse los tipos primitivos de datos. Por lo regular el tipo de un atributo no debería ser un concepto complejo del dominio, como *Venta* o *Aeropuerto*. Por ejemplo, el siguiente atributo *actual* de *TPDV* en el tipo *Cajero* de la figura 11.2 no es idóneo porque con su tipo se indica una *TPDV*, que no es un tipo de atributo simple (digamos *Número* o *Cadena*). La forma más conveniente de expresar que un *Cajero* utiliza una *TPDV* es recurrir a una asociación, no a un atributo.

En un modelo conceptual, es preferible que los atributos sean **atributos simples** o **valores puros de datos**.

Entre los tipos comunes de atributos simples más frecuentes se cuentan:

Booleano, Fecha, Numero, Cadena (Texto), Hora

He aquí otros tipos comunes:

Direccion, Color, Geometria (Punto, Rectangulo, ...), Numero telefonico, Numero del Seguro Social,Codigo Universal de Producto (CUP), SKU, CP o codigos postales, tipos enumerados

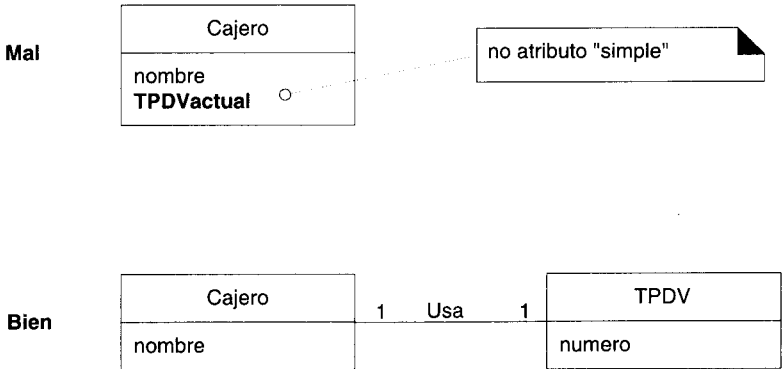


Figura 11.2 Relación con asociaciones, no con atributos.

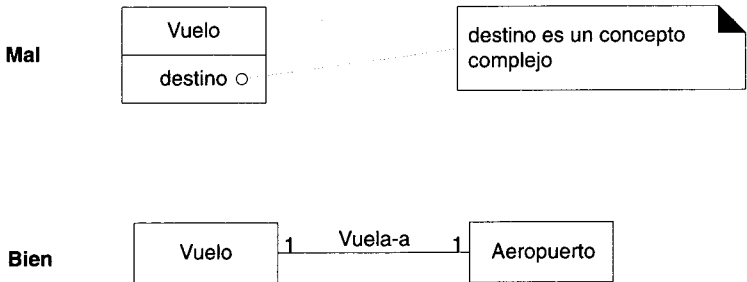


Figura 11.3 No represente como atributos los conceptos complejos de dominio; use asociaciones.

Repetimos un ejemplo anterior: una confusión frecuente consiste en modelar como atributo un concepto complejo del dominio. Así, un aeropuerto de destino no es una cadena de caracteres en realidad; es una realidad compleja que ocupa muchos kilómetros cuadrados de espacio. Por tanto, deberíamos relacionar *Vuelo* con *Aeropuerto* a través de una asociación y no con un atributo, como se indica en la figura 11.3.

Relacione conceptos con una asociación, no con un atributo.

11.4.2 Comparación entre análisis y diseño: ¿qué decir de los atributos en código?

La restricción de que los atributos del modelo conceptual sean únicamente tipos de datos simples *no* significa que los de C++, Java o Smalltalk (miembros de datos, variables de instancias) deban ser sólo de tipos primitivos de datos simples. El modelo conceptual se centra en afirmaciones analíticas puras sobre un dominio del problema, no en entidades de software.

Más adelante, durante las fases del diseño y construcción, veremos que las asociaciones entre los objetos expresados en el modelo conceptual a menudo se implementarán como atributos que apuntan a otros tipos complejos. Pero ésta no es más que una de muchas posibles soluciones de diseño para implementar una asociación; por tanto, la decisión habrá de aplazarse durante la fase de análisis. La etapa de análisis no es el momento de tomar prematuramente decisiones concernientes al diseño.

11.4.3 Valores puros de datos

En términos más generales, los atributos deberían ser **valores puros de datos** (o **Tipos de Datos**, en el lenguaje UML), en los cuales la identidad única no es significativa (dentro del contexto de nuestro modelo o sistema) [Rumbaugh91]. Por ejemplo, no es (generalmente) significativo distinguir entre:

- Instancias aisladas del *Numero* 5.
- Instancias aisladas de la *Cadena* "gato".
- Instancias aisladas de *NumeroTelefonico* que contengan el mismo número.
- Instancias aisladas de *Direccion* que contengan la misma dirección.

En cambio, sí es significativo distinguir (por identidad) entre dos instancias aisladas de una *Persona* cuyos nombres sean "Jill Smith", porque ambas instancias pueden representar diferentes individuos con un mismo nombre.

Por lo que respecta al software, hay pocas situaciones donde compararíamos la dirección de memoria de las instancias de *Numero*, *Cadena*, *NumeroTelefonico* o *Direccion*; sólo las comparaciones basadas en valores son relevantes. En cambio, es posible comparar las direcciones de memoria de las instancias *Persona* y distinguirlas, aun cuando tuvieran los mismos valores de atributos, por ser importante su identidad única.

Un elemento de un tipo puro de datos puede ejemplificarse en la sección de otro concepto destinada a los atributos, aunque también es aceptable modelarlo como un concepto diferente.

Los valores puros de datos también se denominan **objetos de valor**.

El concepto de valor puro es sutil. Una regla práctica consiste en aplicar la prueba básica de los tipos de atributos "simples": convertirlos en atributo si espontáneamente los concebimos como número, cadena, booleano, fecha u hora (y así sucesivamente); de lo contrario, representarlos como un concepto aparte.

En caso de duda, defina algo como concepto aislado y no como atributo.

11.4.4 Deterioro del diseño: ningún atributo debe incluirse como llave foránea

Los atributos no deberían servir para relacionar conceptos en el modelo conceptual. La violación más frecuente de esta regla consiste en agregar un tipo de **atributo de llave foránea**, lo cual suele hacerse con los diseños de bases de datos relacionales, a fin de asociar dos tipos. Por ejemplo, en la figura 11.4 el atributo *NumeroTPDVactual* en el tipo *Cajero* no es conveniente, porque su propósito es relacionar *Cajero* con un objeto *TPDV*. La mejor manera de expresar que un *Cajero* usa una *TPDV* consiste en recurrir a la asociación, no a un atributo de llave foránea. Una vez más, recuerde el lector relacionar los tipos a través de una asociación y no con un atributo.

Hay muchas formas de relacionar los objetos —las llaves foráneas son una de tantas—; además para evitar el **deterioro** del diseño, deberíamos posponer hasta la fase del diseño cómo vamos a implementar la relación.

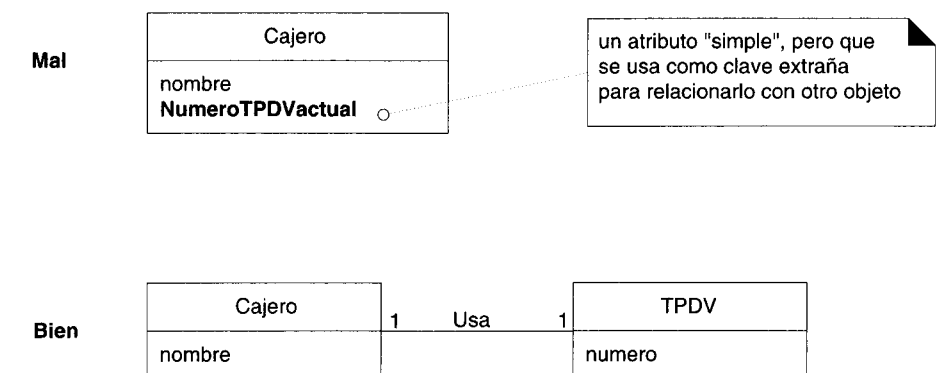


Figura 11.4 No utilice los atributos como claves extrañas.

11.5 Tipos de atributos no primitivos

En el modelo conceptual el tipo de un atributo puede expresarse como no primitivo por sí mismo. Así, en el sistema del punto de venta hay un Código universal de producto (CUP). Suele considerársele como un número. ¿Debería entonces representarse como un tipo no primitivo? Aplique la siguiente directriz:

Represente como tipo no primitivo lo que inicialmente puede considerarse un tipo primitivo de datos (un número o una cadena, por ejemplo), si:

- Se compone de secciones independientes.
 - número telefónico, nombre de persona
- Normalmente se asocian a él operaciones como el análisis o la validación.
 - número del seguro social
- Posee otros atributos.
 - el precio promocional podría tener fecha de inicio y de terminación
- Es una cantidad con una unidad.
 - el importe del pago tiene una unidad monetaria

Al aplicar las directrices anteriores a los atributos del modelo conceptual del punto de venta, se obtiene el siguiente análisis:

- El código *cup* debería ser un tipo *CUP* no primitivo, porque puede ser validado verificando la suma y porque puede poseer otros atributos (por ejemplo, el fabricante que lo asignó).
- Los atributos de *precio* y *cantidad* deberían ser los tipos no primitivos *Cantidad*, por ser cantidades en una unidad monetaria.
- El atributo *direccion* debería ser un tipo no primitivo *Direccion*, porque consta de secciones independientes.

Los tipos *CUP*, *Direccion* y *Cantidad* son valores puros de datos (la identidad única no es significativa); así que podemos mostrarlos en la sección de la casilla dedicada a los atributos en vez de relacionarse con una línea de asociación.

11.5.1 ¿Dónde mostrar los tipos no primitivos de atributos y los valores puros?

¿Deberíamos mostrar el tipo de *CUP* como concepto aparte en un modelo conceptual? La respuesta es: depende de lo que queramos destacar en el diagrama. Por ser el CUP

un valor puro de datos (la identidad única no es importante), podría presentarse en la sección dedicada a atributos en la casilla de conceptos, como se advierte en la figura 11.5. Pero por no ser un tipo primitivo con sus propios atributos y asociaciones, tal vez sería interesante mostrarlo como concepto en su propia caja. No existe en este caso una respuesta correcta; dependerá de la manera en que vamos a utilizar el modelo conceptual como herramienta de comunicación y también dependerá de la importancia del concepto en el dominio.

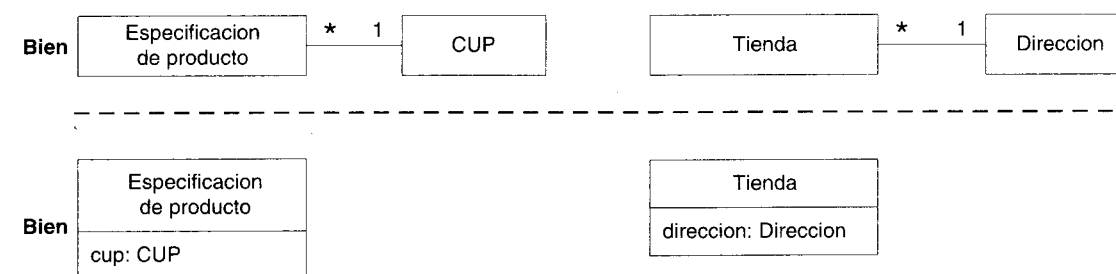


Figura 11.5 Si el tipo de atributo es un valor puro de datos, puede mostrarse en la sección de atributos.

Un modelo conceptual es una herramienta de la comunicación; las decisiones sobre lo que debería mostrarse han de tomarse teniendo presente eso.

11.6 Modelado de cantidades y unidades de los atributos

En términos informales, el importe de un *Pago* puede ser representando como un *Número*. Pero, en general, éste no es un esquema robusto ni flexible porque las *unidades* de un número a menudo son importantes. Considere:

- moneda
- velocidad

Por ejemplo suele requerirse convertir las unidades (por ejemplo, las conversiones del sistema inglés al sistema métrico). Suponiendo que el software del punto de venta esté destinado al mercado internacional, habría que conocer la unidad monetaria de los pagos.

La solución consiste en representar *Cantidad* como un concepto distinto, con una *Unidad* asociada. Y como se supone que las cantidades son valores puros de datos (la identidad única carece de importancia), es aceptable limitarse a presentarlas en la sección de la casilla destinada a los tipos (véase la figura. 11.6).

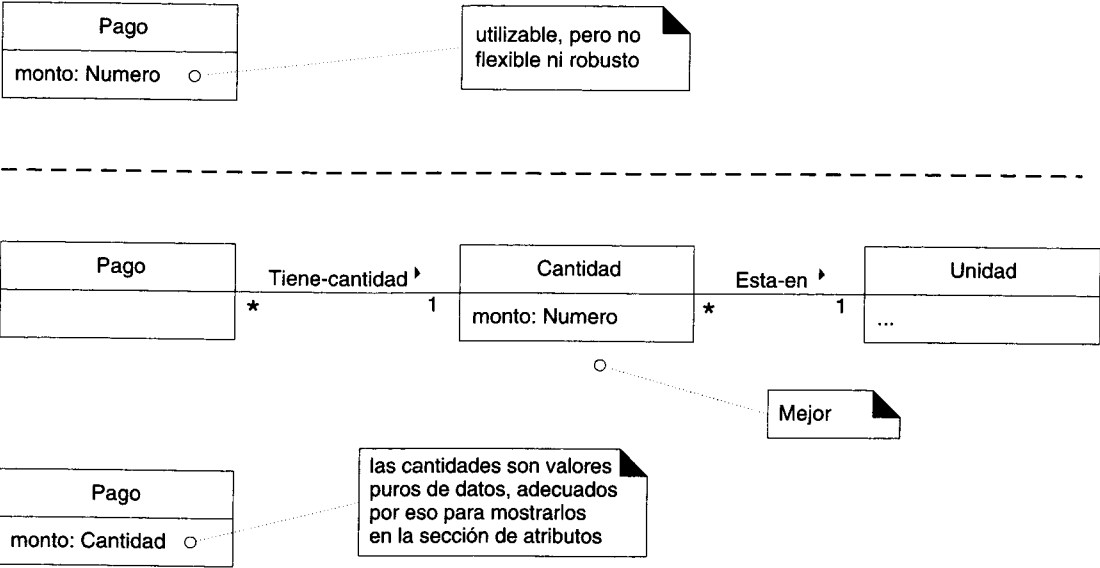


Figura 11.6 Modelado de cantidades.

11.7 Atributos del sistema del punto de venta

Es necesario producir una lista de atributos para los conceptos del dominio del punto de venta. Debería estar reservada primordialmente a los requerimientos y a las simplificaciones en cuestión: los casos simplificados *Comprar Productos: versión 1*.

La lectura llama claramente algunos atributos:

- especificación de los requerimientos
- casos de uso en cuestión
- documentos de simplificaciones, clarificaciones y suposiciones

Por ejemplo, normalmente es necesario registrar la fecha y la hora de una venta. De ahí que el concepto *Venta* requiera los atributos *fecha* y *hora*.

Otros atributos no son evidentes y posiblemente no se los identifique en el análisis. Esto es aceptable: durante las fases de diseño y construcción descubriremos e iremos incorporando el resto de los atributos.

En la siguiente sección se resumen las opciones de atributos indicadas por los casos actuales de uso.

11.8 Atributos en el modelo del punto de venta

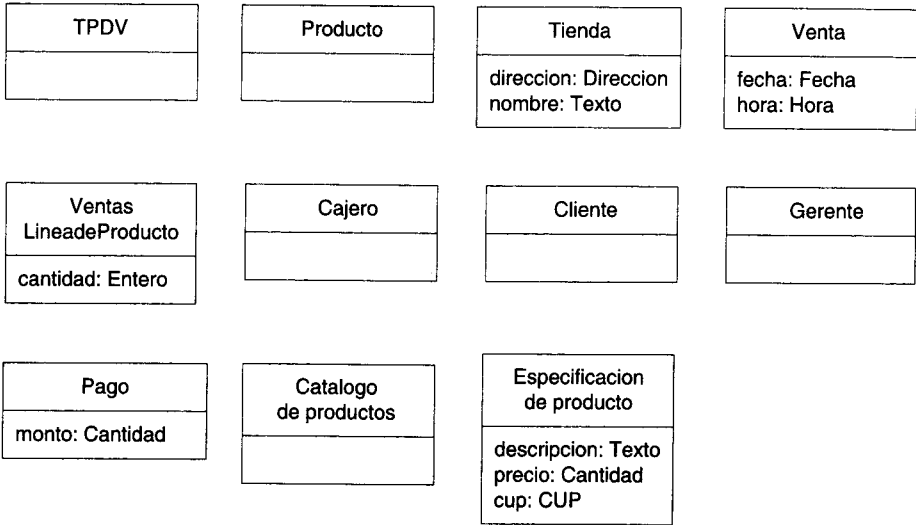


Figura 11.7 Modelo conceptual que muestra los atributos.

11.8.1 Explicación de los atributos TPDV

<i>Pago</i>	<i>importe</i> : hay que capturar un monto (llamado también “importe ofrecido”) para determinar si se dio un pago suficiente y calcular el cambio.
<i>EspecificaciondeProducto</i>	<i>descripcion</i> : para incluir una descripción en un desplegue. <i>CUP</i> : Para consultar EspecificaciondeProducto, una vez capturado un CUP, es necesario relacionarlos con un CUP. <i>precio</i> : para calcular el total de las ventas y mostrar el precio de la línea del producto.
<i>Venta</i>	<i>fecha, hora</i> : el recibo es un informe escrito de una venta. Normalmente contiene la fecha y la hora de la venta.

VentasLineadeProducto *cantidad*: para registrar la cantidad capturada, cuando hay más de un elemento en la línea del producto (por ejemplo, *cinco* paquetes de pañuelos desechables).

Tienda *dirección, nombre*: el recibo requiere el nombre y la dirección de la tienda.

11.9 Multiplicidad entre VentasLineadeProducto y Producto

Es posible que un cajero reciba un grupo de productos afines (seis paquetes de pañuelos desechables, por ejemplo), que introduzca una vez el código universal de producto y luego una cantidad (seis, por ejemplo). En consecuencia, un *VentasLineadeProducto* puede estar asociado a más de una instancia de cada producto. La cantidad que captura el cajero puede quedar registrada como atributo de *VentasLineadeProducto* (figura 11.8). Sin embargo, también puede calcularla a partir del valor real de multiplicidad de la relación; así que puede caracterizarse como un **atributo derivado**, el cual puede ser deducido de otra información. En el lenguaje UML, un atributo de esta clase se denota con el símbolo “/”.

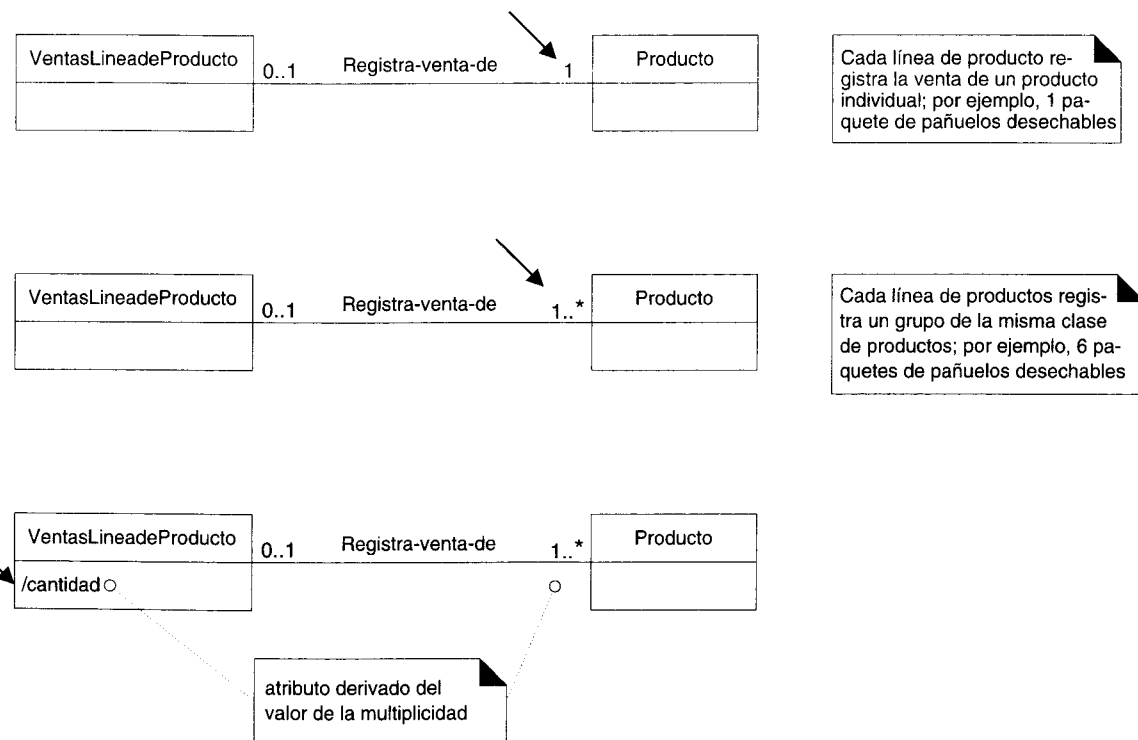


Figura 11.8 Registrar la cantidad de productos vendidos en una línea de producto.

11.10 Modelo conceptual del punto de venta

Al combinar los conceptos, asociaciones y atributos que descubrimos en la investigación anterior, obtenemos el modelo de la figura 11.9.

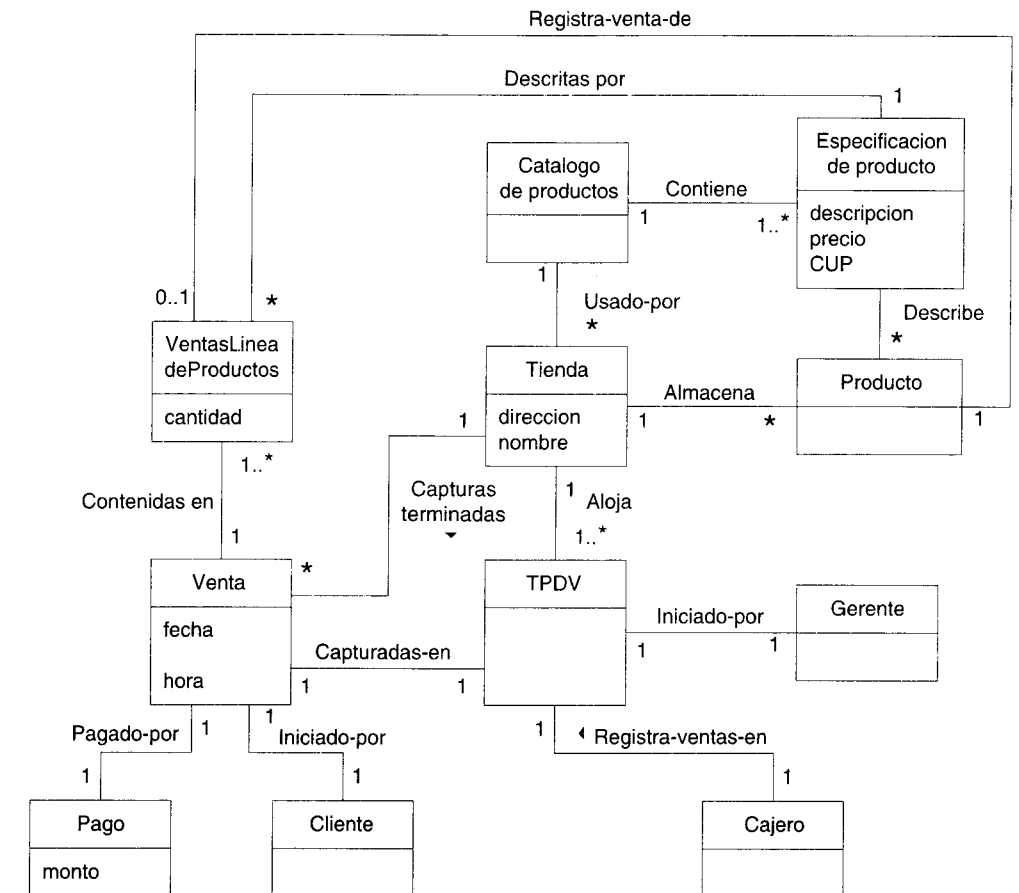


Figura 11.9 Modelo conceptual del dominio del punto de venta.

11.11 Conclusión

Hemos creado un modelo conceptual relativamente útil del dominio de la aplicación al punto de venta. No existe un modelo apropiado para todos los casos o circunstancias. Todos ellos no son más que aproximaciones al dominio que intentamos entender. Un buen modelo conceptual capta las abstracciones esenciales y la información indispensable para comprender el dominio dentro del contexto de los requerimientos actuales; nos facilita además conocer el dominio: sus conceptos, su terminología y sus relaciones.

REGISTRO DE LOS TÉRMINOS EN EL GLOSARIO

Objetivos

- Expresar términos en un glosario.

12.1 Introducción

El glosario es un documento simple en el cual se definen términos. En el presente capítulo ofrecemos un ejemplo de glosario aplicado al dominio del punto de venta.

12.2 Glosario

El **glosario** o **diccionario modelo** (semejante a un diccionario de datos) incluye y define todos los términos que requieren explicación para mejorar la comunicación y aminorar el riesgo de malos entendidos.

Un significado uniforme y compartido resulta extremadamente importante durante el desarrollo de las aplicaciones, sobre todo cuando muchos miembros del equipo intervienen en el proyecto.

12.3 Actividades y dependencias

El glosario se crea originalmente durante la fase de Planeación y Elaboración conforme vayan generándose los términos; después va perfeccionándose continuamente en cada ciclo de desarrollo al aparecer nuevos vocablos. Por lo regular se realiza junto con la especificación de requerimientos, los casos de uso y el modelo conceptual. Darle mantenimiento es una actividad permanente a lo largo de todo el proyecto.

12.3.1 Reglas y restricciones del dominio

El glosario es un documento muy útil donde se registran las reglas del dominio de la empresa, las restricciones y otros puntos, aunque aquí no examinaremos este aspecto. He aquí otros artefactos en que se registra esta clase de información: los casos de uso, los contratos (que veremos en un capítulo subsecuente) y la incorporación de notas de restricción a los elementos (los conceptos, entre ellos) a los que se aplica la regla o la restricción.

12.4 Ejemplo de glosario aplicado al sistema del punto de venta

No existe un formato oficial de este tipo de glosario. Anexamos aquí una muestra:

Término	Categoría	Comentarios
Comprar productos	caso de uso	Descripción del proceso de un cliente que compra productos en una tienda.
EspecificaciondeProducto.descripcion: Texto	atributo	Descripción breve de un producto en una venta, junto con su <i>EspecificaciondeProducto</i> asociada.
Producto	tipo	Un producto para venderse en una <i>Tienda</i> .
Pago	tipo	Un pago en efectivo.
EspecificaciondeProducto.precio: Cantidad	atributo	El precio de un producto en una venta, junto con su <i>EspecificaciondeProducto</i> asociada.
VentasLineadeProducto.cantidad: Entero	atributo	La cantidad comprada de un tipo de Producto.
Venta	tipo	Una transacción de ventas.

Término	Categoría	Comentarios
VentasLineadeProducto	tipo	Una línea de productos de un producto particular comprado en una <i>Venta</i> .
Tienda	tipo	El lugar donde se realiza la venta de productos.
Venta.total: Cantidad	atributo	El gran total de la <i>Venta</i> .
Pago.monto: Cantidad	atributo	El monto que el cliente ofrece o presenta para el pago.
EspecificaciondeProducto.cup: CUP	atributo	El código universal de producto del Producto y su <i>EspecificaciondeProducto</i> .

UNIVERSIDAD DE LA REPUBLICA
FACULTAD DE INGENIERIA
DEPARTAMENTO DE
DOCUMENTACION Y BIBLIOTECA
MONTEVIDEO - URUGUAY

COMPORTAMIENTO DE LOS SISTEMAS: DIAGRAMA DE LA SECUENCIA DEL SISTEMA

Objetivos

- Identificar los eventos y las operaciones del sistema.
- Crear el diagrama de la secuencia del sistema para los casos de uso.

13.1 Introducción

El diagrama de la secuencia de un sistema muestra gráficamente los eventos que fluyen de los actores al sistema. En este capítulo describiremos la forma de elaborarlos.

La creación de los diagramas de la secuencia de un sistema forma parte de la investigación para conocer el sistema; se incluye, pues, dentro del modelo de análisis.

El UML ofrece una notación con los diagramas de la secuencia que muestran gráficamente los eventos que pasan de los actores al sistema.

13.2 Actividades y dependencias

Los diagramas de la secuencia de un sistema se preparan durante la fase de análisis de un ciclo de desarrollo. Su creación depende de la formulación previa de los casos de uso.

13.3 Comportamiento del sistema

Antes de iniciar el diseño lógico de cómo funcionará una aplicación de software, es necesario investigar y definir su comportamiento como una “caja negra”. El **comportamiento del sistema** es una descripción de lo *que* hace, sin explicar la manera en que lo hace. Una parte de la descripción es un diagrama de la secuencia del sistema.

13.4 Diagramas de la secuencia del sistema

Los casos de uso indican cómo los actores interactúan con el sistema de software que es lo que en realidad deseamos crear. Durante la interacción un actor genera eventos dirigidos a un sistema, solicitando alguna operación a cambio. Por ejemplo, cuando un cajero introduce un código universal de producto de un artículo, está pidiendo al sistema TPDV registrar la compra. Con el evento de esa petición se inicia una operación del sistema.

Conviene aislar y explicar gráficamente las operaciones que un actor solicita a un sistema, porque contribuyen de manera importante a entender el comportamiento del sistema. El UML incluye entre su notación los **diagramas de la secuencia** que dan una descripción gráfica de las interacciones del actor y de las operaciones a que da origen.

El **diagrama de la secuencia de un sistema** es una representación que muestra, en determinado escenario de un caso de uso,¹ los eventos generados por actores externos, su orden y los eventos internos del sistema. A todos los sistemas se les trata como una caja negra; los diagramas se centran en los eventos que trasciende las fronteras del sistema y que fluyen de los actores a los sistemas.

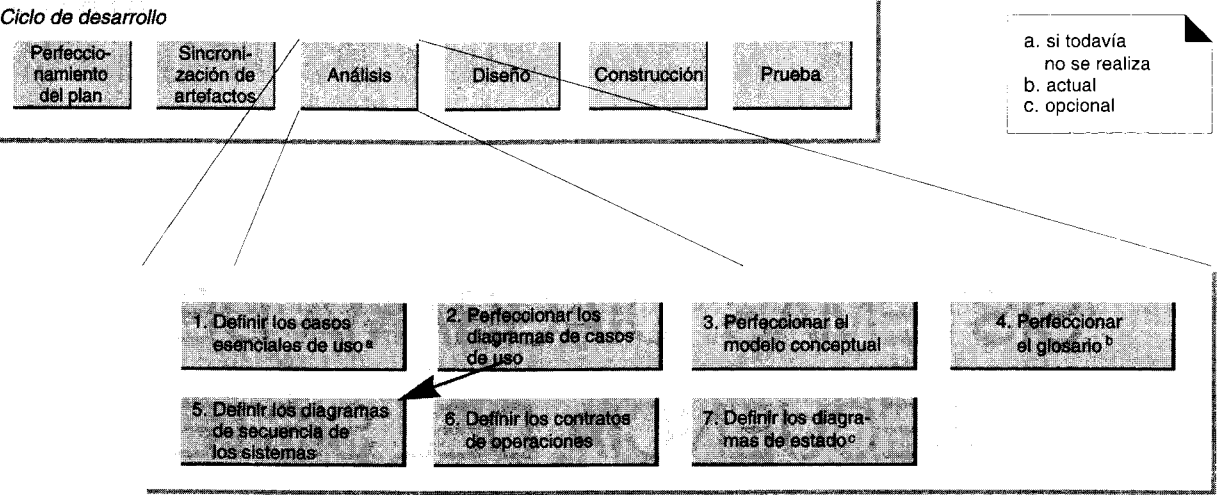
El diagrama de la secuencia de un sistema debería prepararse para el curso normal de los eventos de un caso de uso —y posiblemente también de otros—, teniendo en cuenta los cursos opcionales más interesantes.

13.5 Ejemplo de un diagrama de la secuencia de un sistema

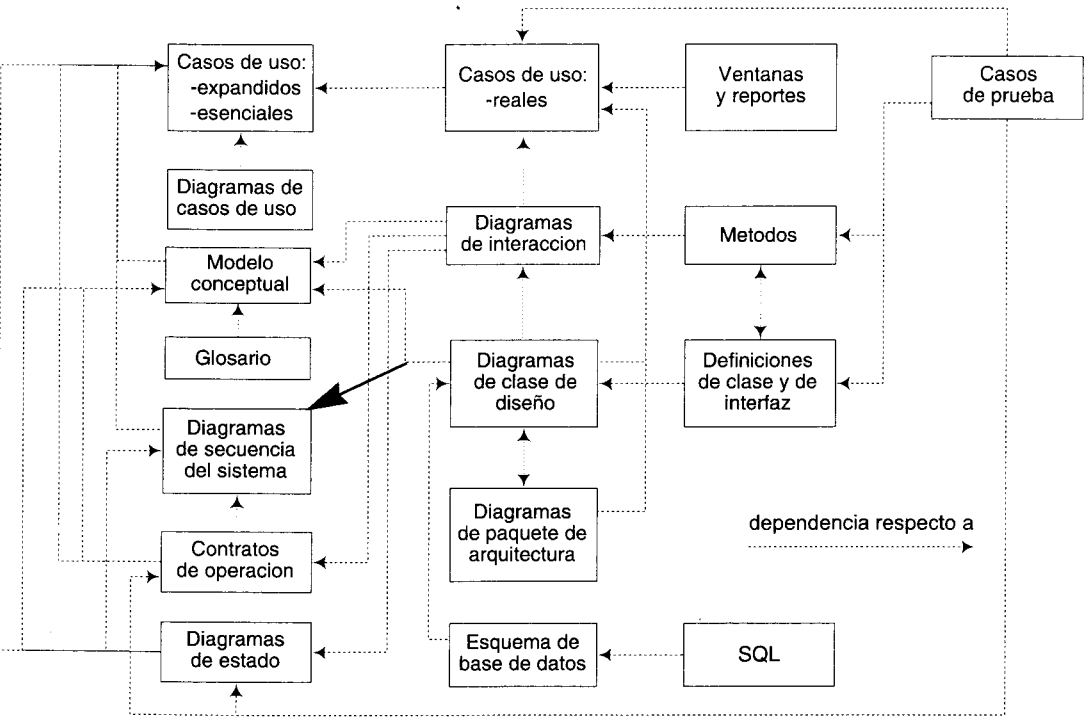
El **diagrama de la secuencia de un sistema** describe, en el curso particular de los eventos de un caso de uso, los actores externos que interactúan directamente con el sistema (como caja negra) y con los eventos del sistema generados por los actores (figura 13.1). En el diagrama el tiempo avanza hacia abajo, y el ordenamiento de los eventos debería seguir el orden indicado en el caso de uso.

¹ El escenario de un caso de uso es una instancia o trayectoria realizada por medio del uso: un ejemplo real de su ejecución.

Ciclo de desarrollo



Actividades de la fase de análisis dentro de un ciclo de desarrollo.



Dependencias de los artefactos durante la fase de construcción.